



Proyecto DEMETER

Plataforma Integral de Monitorización y Gestión
para Investigación Agrícola

IoT · Firmware ESP32 · Backend FastAPI · LIMS

Documentación Técnica del Sistema

Autor: Daniel Montero Fernández

Contacto: danmonfer@us.es

Institución: Universidad de Sevilla

Versión: 1.0

Fecha: Febrero de 2026

Estado: En desarrollo activo

Este documento es confidencial y de uso interno del proyecto.

Índice

Introducción y Visión del Proyecto	5
0.1 Motivación y Problema a Resolver	5
0.2 Objetivos del Sistema	5
0.2.1 Objetivo General	5
0.2.2 Objetivos Específicos	5
0.3 Descripción General del Sistema	5
1 Protocolo de Comunicación Demeter	7
1.1 Propósito y Filosofía de Diseño	7
1.2 Modelos Pydantic: Bridging entre Binario y JSON	7
1.2.1 Diseño de Modelos Discriminados	7
1.2.2 Auto-descubrimiento y Clasificación Automática	8
1.2.3 Conversión Binaria: Solo los Valores	8
1.3 Comunicación HTTP y WebSocket en el Sistema	8
1.3.1 Flujo de un Comando desde el Navegador	8
1.3.2 Flujo de Telemetría en Tiempo Real	9
1.4 Estructura de una Trama	9
1.5 Catálogo de Mensajes	10
1.6 Codificación de Datos	10
1.6.1 Temperatura y Humedad (Punto Fijo $\times 100$)	10
1.6.2 Duraciones en Secuencias (32 bits Little Endian)	10
1.7 Ejemplos de Tramas Reales	10
1.7.1 Telemetría de Sensor (TEMP_HUM_REPORT)	10
1.7.2 Comando de Actuación (SET_GPIO)	11
1.7.3 Inicio de Handshake (SYN)	11
1.8 Algoritmos de Serialización y Deserialización	11
1.8.1 Serialización	11
1.8.2 Deserialización y Validación	11
1.8.3 Manejo de Errores	11
1.9 Handshake TCP-like	12
1.10 Flujo Extremo a Extremo	12
1.11 Consideraciones de Tiempo y Saturación	12
1.11.1 Tiempos de Transmisión	12
1.11.2 Ciclo Energético del Nodo Sensor	12
1.11.3 Limitaciones Conocidas	13
2 Arquitectura del Firmware	14
2.1 Visión General	14
2.2 Los Tres Tipos de Nodo	14
2.2.1 Nodo Sensor	14
2.2.2 Nodo Actuador	14

2.2.3	Nodo Gateway	15
2.3	Arquitectura Interna en Capas	15
2.3.1	El Orquestador del Sistema	15
2.4	El Motor de Protocolo	16
2.4.1	Responsabilidades del Motor	16
2.4.2	Pipeline de Recepción	16
2.5	Estrategias de Transporte	16
2.5.1	Transporte ESP-NOW	16
2.5.2	Transporte UART (Serial)	17
2.5.3	Transporte Mixto (Gateway)	17
3	Servicio de Borde: Raspberry Pi	18
3.1	Misión y Posición en la Arquitectura del Sistema	18
3.2	Contenedorización y Acceso a Recursos del Sistema	18
3.3	El Motor de Concurrencia: asyncio	19
3.4	El Módulo del Protocolo Demeter en la Raspberry Pi	19
3.5	Comunicación con el Backend: El Canal WebSocket	20
3.6	El Command Dispatcher: Adaptador JSON ↔ Binario	20
3.7	Device Manager: La Caché Local de Topología	21
4	Servidor Central: Backend en Google Cloud	22
4.1	Visión General del Servidor	22
4.2	Arquitectura de Contenedores Docker	22
4.3	Redis: Bus de Mensajes y Caché	23
4.3.1	Las Cuatro Categorías de Datos en Redis	23
4.3.2	Canales Pub/Sub	23
4.4	PostgreSQL: Persistencia y Esquema de Datos	23
4.5	El Worker Matemático	24
4.6	El Secuenciador	24
4.7	El Backend FastAPI	25
4.8	El Frontend React	25
4.9	Documentación de la API REST	25
4.9.1	Sistema (/api/system)	25
4.9.2	Autenticación (/api/auth)	26
4.9.3	Comandos (/api/command)	26
4.9.4	Descubrimiento (/api/discovery)	26
4.9.5	Historial de Telemetría (/api/history)	26
4.9.6	LIMS — Plantas y Experimentos (/api/lims)	27
4.9.7	Análisis Científico (/api/analysis)	27
4.9.8	SDK Científico (/api/sdk)	27
4.9.9	Exportación (/api/export)	27
4.10	El Canal WebSocket	27
5	SDK Científico de Python — demeter_sdk	29
5.1	Propósito y Filosofía	29
5.2	Arquitectura Modular	29
5.3	Instalación y Configuración	30
5.4	Obtención de Datos: Módulo Fetcher	30
5.5	Pipeline ETL: Módulo Transform	30
5.6	Cálculos Agronómicos: Módulo Science	31
5.6.1	Termodinámica del Aire Húmedo	31
5.6.2	Fenología y Tiempo	34

5.6.3	Estadística y Detección de Anomalías	35
5.7	El Helper <code>enrich()</code> : Enriquecimiento en Una Llamada	35
5.8	Visualización Científica: Módulo Viz	36
5.9	Exportación Científica: Módulo Export	37
5.9.1	CSV	37
5.9.2	Excel Multi-hoja	37
5.10	Suite Completa — Un Solo Comando	38
5.11	Ejemplo de Sesión Completa con Jupyter	38
5.12	Referencia Rápida de Parámetros	39
A	Diagramas del Protocolo Binario Demeter	40
B	Arquitectura del Servicio Raspberry Pi	50
C	Arquitectura del Backend	56

Introducción y Visión del Proyecto

0.1 Motivación y Problema a Resolver

La agricultura de precisión y la automatización del cuidado botánico representan uno de los campos de mayor proyección en la intersección entre la electrónica, el software y las ciencias naturales. En entornos como invernaderos de investigación, la falta de herramientas de monitorización continua obliga a los investigadores a realizar lecturas manuales, perder datos valiosos durante la noche y tomar decisiones de riego basadas en la intuición.

El Proyecto Demeter nace para resolver este problema. Propone un ecosistema tecnológico completo que automatiza la recolección de variables ambientales mediante sensores, asegura la trazabilidad física de cada sujeto de prueba y facilita el análisis científico avanzado mediante herramientas modernas y de bajo coste.

0.2 Objetivos del Sistema

0.2.1 Objetivo General

Desarrollar una plataforma IoT integral que permita monitorizar, controlar y analizar entornos de cultivo, proporcionando trazabilidad científica completa de las plantas y los experimentos asociados.

0.2.2 Objetivos Específicos

- **Automatización IoT:** Desplegar una red de sensores de bajo coste y alta fiabilidad que recolecte variables ambientales de forma continua y sin intervención humana.
- **Control Remoto:** Permitir la actuación sobre los elementos físicos del entorno (bombas de riego, electroválvulas, ventanas) desde cualquier punto con acceso a internet.
- **Trazabilidad LIMS:** Implementar un sistema de gestión de información de laboratorio que mantenga un registro estructurado de cada planta y los experimentos en los que participa.
- **Análisis Científico:** Dotar a los investigadores de herramientas (SDK, gráficas, informes exportables) para extraer y analizar datos sin depender del equipo de soporte.

0.3 Descripción General del Sistema

El sistema Demeter se articula en cuatro módulos interconectados que cubren el ciclo completo desde la adquisición de datos hasta su interpretación científica:

1. **Capa de Campo (Firmware ESP32):** Red de microcontroladores ESP32 comunicados mediante ESP-NOW. Tres roles diferenciados: sensores, actuadores y gateway.
2. **Capa de Borde (Servicio Raspberry Pi):** Servicio Python que recibe las tramas del gateway por UART y las reenvía al servidor central.

3. **Capa de Servidor (Backend FastAPI):** API REST y WebSocket asíncrona que gestiona la persistencia en PostgreSQL y la publicación de datos en tiempo real.
4. **Capa de Visualización (Frontend React):** SPA que muestra datos en vivo, permite crear secuencias de riego y gestionar experimentos científicos (LIMS).

Capítulo 1

Protocolo de Comunicación Demeter

1.1 Propósito y Filosofía de Diseño

El Protocolo Demeter es un protocolo de comunicación binario diseñado específicamente para la red de campo del sistema. Los protocolos estándar de texto (JSON, CSV) son demasiado pesados para microcontroladores con memoria limitada y canales de radio estrecho. Un paquete JSON con temperatura y humedad puede ocupar entre 40 y 80 bytes. La misma información en el Protocolo Demeter ocupa exactamente **10 bytes**.

Decisión de Diseño

Las decisiones de diseño que guían el protocolo son: **compacidad** (cada bit cuenta), **determinismo** (la longitud de cada trama es conocida antes de recibirla), **robustez** (verificación de integridad en toda trama) y **extensibilidad** (hasta 254 tipos de mensaje sin modificar la estructura base).

1.2 Modelos Pydantic: Bridging entre Binario y JSON

Una aportación central de la arquitectura Demeter es que el protocolo no es exclusivamente binario: las mismas definiciones de mensaje coexisten en **dos representaciones**, gestionadas mediante **modelos Pydantic** validados y tipados:

1. **Representación JSON**: usada en la comunicación entre el Frontend y el Backend por HTTP y WebSocket. Los campos viajan con sus nombres completos (e.g. "type", "target_id", "pin", "value").
2. **Representación Binaria**: usada en la comunicación de red de campo (UART entre Raspberry Pi y Gateway, y ESP-NOW entre nodos). Solo los valores numéricos viajan en la trama; los nombres de campo *no existen en el paquete*, reduciéndolo a su mínima expresión.

1.2.1 Diseño de Modelos Discriminados

Cada tipo de comando del protocolo tiene su propio modelo Pydantic con un campo **type** que actúa como **discriminador de unión**. Esto permite que al deserializar un JSON, Pydantic identifique automáticamente de qué tipo de mensaje se trata y lo valide con el esquema apropiado:

Listing 1.1: Estructura conceptual de un modelo discriminado

```
{
  "type": "set_gpio", <-- discriminador: identifica el subtipo exacto
  "target_id": 3, <-- nodo destino
```

```
"pin": 4, <-- pin a controlar
"value": 1, <-- 1=ON, 0=OFF
"duration_ms": 15000 <-- tiempo de activacion (ms)
}
```

La clave del diseño es que el campo **type** es unívoco por subtipo. Cuando el Backend recibe este JSON (desde el Frontend o la API), instancia mediante `TypeAdapter[AnyDemeterCommand]` el modelo correcto sin lógica condicional manual.

1.2.2 Auto-descubrimiento y Clasificación Automática

El uso del patrón de **unión discriminada** con Pydantic permite que cualquier capa del sistema clasifique un mensaje desconocido sin necesidad de *switch-case* explícitos: la librería inspecciona el campo **type** y devuelve la instancia del modelo correcto, ya validada y tipada. Esto tiene tres consecuencias importantes:

- **Validación zero-cost:** si un campo falta o tiene tipo incorrecto, Pydantic lanza una excepción detallada antes de que el sistema procese nada.
- **Serialización simétrica:** `model.model_dump_json()` produce el JSON que el Frontend espera; `model.to_binary()` produce la trama que el ESP32 espera. La lógica de transformación reside en el modelo, no en el controlador.
- **Reutilización del esquema:** el mismo `Common/schemas.py` es importado por el Backend (Python) y puede referenciarse para generar SDKs cliente, documentación automática (OpenAPI) o tests de integración.

1.2.3 Conversión Binaria: Solo los Valores

La diferencia fundamental entre JSON y binario es que en la trama física **los nombres de los campos no existen**. El orden y tamaño de cada byte es implícito al tipo de comando (definido por el `CMD_ID` de la cabecera). Por ejemplo, para un `SET_GPIO` el payload es exactamente 3 bytes: `[PIN, VALUE, FLAGS]`.

La serialización binaria rellena cada byte en orden, con el tamaño y signo definidos por el esquema del protocolo (enteros sin signo, punt fijo para float, Little Endian).

1.3 Comunicación HTTP y WebSocket en el Sistema

1.3.1 Flujo de un Comando desde el Navegador

Cuando un operador interactúa con el dashboard activo, los comandos siguen un flujo completamente desacoplado de la red de campo:

1. El **Frontend React** compone un objeto JSON conforme al esquema del modelo Pydantic correspondiente (e.g. `set_gpio`) y lo envía mediante una petición **HTTP POST** a `/api/command`.
2. El **Backend FastAPI** recibe el JSON, lo valida automáticamente con el `TypeAdapter` discriminado, y publica el comando en el canal Redis `demeter:commands`. Devuelve `202 Accepted` inmediatamente.
3. El **WS Dispatcher** (tarea asyncio en el Backend) consume el mensaje de Redis y lo retransmite por WebSocket a la Raspberry Pi, identificada como `raspberry_gateway`.
4. La **Raspberry Pi** recibe el JSON por WebSocket, lo convierte a trama binaria mediante el mismo modelo Pydantic en Python, y lo envía por UART al Gateway ESP32.

5. El **Gateway ESP32** reenvía la trama binaria por ESP-NOW al nodo actuador destino, que ejecuta la acción y confirma con `CMD_ACK`.

1.3.2 Flujo de Telemetría en Tiempo Real

El camino inverso (del sensor al dashboard) funciona de forma reactiva:

1. El **Nodo Sensor** emite tramas binarias por ESP-NOW en el intervalo configurado.
2. El **Gateway ESP32** las reenvía por UART a la Raspberry Pi.
3. La **Raspberry Pi** las decodifica (Protocolo Demeter en Python), construye un JSON y lo publica por WebSocket hacia el Backend.
4. El **Backend** almacena la medición en Redis (caché 60 s) y en PostgreSQL (persistencia), y hace *broadcast* por WebSocket a todos los clientes Frontend suscritos.
5. El **Frontend** recibe el push y actualiza los gráficos sin ninguna petición adicional por parte del usuario.

Decisión de Diseño

Separación de canales: Los comandos viajan por **HTTP POST** (fiables, con respuesta de estado). La telemetría viaja por **WebSocket** (baja latencia, push). Para la Raspberry Pi, el canal WebSocket es exclusivo y bidireccional. Para el Frontend, el WebSocket es de solo lectura; sus comandos siempre van por HTTP.

1.4 Estructura de una Trama

Toda comunicación sigue el mismo formato de cabecera fija seguida de un payload variable:

Cuadro 1.1: Campos de la cabecera Demeter

Campo	Tamaño	Descripción
SYNC	1 byte	Siempre 0xAA. Marca el inicio de cualquier trama válida.
FLAGS	1 byte	Reservado. Actualmente 0x00.
SRC_ID	1 byte	Identificador numérico del nodo emisor (1–253).
DST_ID	1 byte	Destino: 0x00=servidor, 0xFF=difusión.
CMD_ID	1 byte	Tipo de mensaje (ver catálogo).
LENGTH	1 byte	Número de bytes del payload (0–255).
PAYLOAD	N bytes	Datos específicos del tipo de mensaje, formato Little Endian.
CRC	1 byte	Suma módulo 256 de FLAGS hasta fin del payload.

Tamaños: mínimo 7 bytes (sin payload) · máximo teórico 262 bytes · típico 10–14 bytes.

1.5 Catálogo de Mensajes

Cuadro 1.2: Catálogo completo de comandos del Protocolo Demeter v2

CMD_ID	Nombre	Payload (bytes)	Descripción
0x01	PING	0	Keep-alive. Espera ACK.
0x02	ACK	0–1	Confirmación con contexto de sesión opcional.
0x03	NACK	1	Rechazo con código de error.
0x04	SYN	0	Inicio del handshake (paso 1/3).
0x05	SYN-ACK	0	Confirmación de disponibilidad (paso 2/3).
0x0A	ROUTE_ADD	7	Registra una nueva ruta en la tabla del Gateway.
0x0B	TEMP_HUM_REPORT	4	Datos de temperatura y humedad (punto fijo $\times 100$).
0x0C	PIN_REPORT	2	Estado de un pin tras ejecución de un comando.
0x0D	SYSTEM_REPORT	8	Modo de operación y nivel de batería en mV.
0x10	SET_GPIO	3	Orden de cambio de estado de un pin.
0x11	SET_PWM	3	Señal analógica (control de velocidad/intensidad).
0x20	GET_SENSORS	0	Solicitud de lectura inmediata de sensores.
0x30	EXEC_SEQUENCE	Variable	Lista de pasos de actuación con temporización.

1.6 Codificación de Datos

1.6.1 Temperatura y Humedad (Punto Fijo $\times 100$)

Los valores se multiplican por 100 y se codifican como enteros con signo de 16 bits en formato Little Endian:

$$T = 24,57^{\circ}\text{C} \rightarrow 2457 = 0x0999 \rightarrow \text{bytes: [99 09]}$$

$$H = 63,20\% \rightarrow 6320 = 0x18B0 \rightarrow \text{bytes: [B0 18]}$$

1.6.2 Duraciones en Secuencias (32 bits Little Endian)

Las duraciones se expresan en milisegundos como entero de 32 bits sin signo:

$$2500\text{ ms} = 0x000009C4 \rightarrow \text{bytes: [C4 09 00 00]}$$

1.7 Ejemplos de Tramas Reales

1.7.1 Telemetría de Sensor (TEMP_HUM_REPORT)

Nodo 2 envía $T=24,57^{\circ}\text{C}$ y $H=63,20\%$ al servidor:

Listing 1.2: Trama TEMP_HUM_REPORT completa

```
Pos: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
Hex: AA 00 02 00 0B 04 99 09 B0 18 CRC
      SYNC FLAGS SRC DST CMD LEN T_Lo T_Hi H_Lo H_Hi
            (2) (0) TH_RPT (4) ---- 24.57 C ---- -63.20%-
```

1.7.2 Comando de Actuación (SET_GPIO)

Servidor ordena al nodo 3 encender el pin 4 (bomba):

Listing 1.3: Trama SET_GPIO

```
Pos: [0] [1] [2] [3] [4] [5] [6] [7] [8] [9]
Hex: AA 00 00 03 10 03 04 01 00 CRC
      SYNC FLAGS SRC DST CMD LEN PIN VAL FLAGS
            (0) (3) SET_IO (3) (4) HIGH (0)
```

1.7.3 Inicio de Handshake (SYN)

Nodo sensor 2 solicita sesión con el Gateway (nodo 1):

Listing 1.4: Trama SYN mínima — 7 bytes

```
Pos: [0] [1] [2] [3] [4] [5] [6]
Hex: AA 00 02 01 04 00 CRC
      SYNC FLAGS SRC DST CMD LEN
            (2) (1) SYN (0)
```

1.8 Algoritmos de Serialización y Deserialización

1.8.1 Serialización

El proceso de serialización transforma los datos de aplicación en una trama binaria completamente estructurada. Toma el comando o reporte, convierte sus campos al formato numérico requerido (punto fijo para temperatura, Little Endian para enteros), construye el payload en un buffer, prepende la cabecera y calcula el CRC.

1.8.2 Deserialización y Validación

El *pipeline* de deserialización actúa como barrera de seguridad: verifica progresivamente el byte de inicio (SYNC), la longitud declarada, el CRC y el destino. Solo si todas las comprobaciones pasan se decodifica el payload y se lanza el evento interno asociado al tipo de mensaje.

1.8.3 Manejo de Errores

Consideración Importante

El protocolo adopta la filosofía de **fallo silencioso**: una trama corrupta no genera ninguna respuesta de error. Esto evita tormentas de retroalimentación ante interferencias de radio. El NACK se reserva para errores de lógica de aplicación (pin no válido, modo IDLE activo).

1.9 Handshake TCP-like

Antes de operar, cada nodo negocia una sesión con el Gateway mediante un triple *handshake*: el emisor envía SYN, espera SYN-ACK con *timeout* de 2 s y hasta 3 reintentos, y confirma con ACK. Solo tras este acuerdo comienzan los reportes.

1.10 Flujo Extremo a Extremo

Una trama parte del Nodo Sensor en formato binario por ESP-NOW, llega al Gateway, se reenvía por UART a la Raspberry Pi, que la decodifica y publica como JSON al Backend. El Backend la persiste en PostgreSQL y distribuye en tiempo real a todos los clientes conectados por WebSocket, con reacción visual a la décima de segundo.

1.11 Consideraciones de Tiempo y Saturación

1.11.1 Tiempos de Transmisión

Cuadro 1.3: Parámetros de tiempo de transmisión del protocolo

Parámetro	Valor
Tiempo de transmisión por trama (10B)	$\approx 0,5$ ms
Latencia extremo a extremo (RF + serial)	3 – 10 ms
Intervalo de reporte del sensor (configurado)	5.000 ms
Ocupación del canal por nodo sensor	$< 0,01\%$

1.11.2 Ciclo Energético del Nodo Sensor

El nodo sensor efectúa un intenso ahorro de recursos invocando Deep Sleep del ESP32 entre reportes. Un nodo configurado con reportes cada 60 s puede operar durante más de un año con una batería de 3000 mAh.

Consideración Importante

Los nodos actuadores y el Gateway deben permanecer activos de forma continua y requieren alimentación estable. Solo el Nodo Sensor puede operar con batería durante periodos prolongados.

1.11.3 Limitaciones Conocidas

Cuadro 1.4: Limitaciones conocidas y decisiones de diseño adoptadas

Limitación	Impacto	Decisión de diseño
Sin confirmación de entrega en ESP-NOW	Una trama puede perderse sin aviso	Aceptable para telemetría; comandos críticos usan ACK de capa de aplicación.
Máx. 250 bytes por trama ESP-NOW	Secuencias limitadas a ≈ 30 pasos por trama	Suficiente para los casos actuales; secuencias largas se pueden particionar.
Sin cifrado nativo	Tramas legibles por cualquier ESP32 cercano	AES-128 en payload es mejora planificada para v3.0.
Tabla de rutas fija (20 entradas)	Sin admisión dinámica sin reiniciar	El comando ROUTE_ADD permite registrar rutas en caliente.

Capítulo 2

Arquitectura del Firmware

2.1 Visión General

El firmware de Demeter implanta una red distribuida de microcontroladores ESP32 que operan en cooperación para cubrir tres funciones complementarias: recolectar datos del entorno, actuar sobre él y servir de puente entre el mundo físico y el sistema de información central.

Cada dispositivo ejecuta exactamente uno de tres roles, definido en tiempo de compilación: **Nodo Sensor**, **Nodo Actuador** o **Nodo Gateway**.

2.2 Los Tres Tipos de Nodo

2.2.1 Nodo Sensor

El Nodo Sensor es la unidad de observación del sistema. Su único propósito es medir el entorno y comunicar esas lecturas al resto de la red.

Capacidades:

- Mide temperatura del aire, humedad relativa y humedad del suelo.
- Opera en modo **Activo** (reporte periódico cada N segundos) o **IDLE** (escucha pasiva a petición del Gateway).
- Soporta datos simulados para desarrollo sin hardware físico.
- Realiza un proceso de establecimiento de sesión (handshake) con el Gateway antes de iniciar el envío de telemetría.
- Entre reportes entra en modo **Deep Sleep**, consumiendo μA de corriente y alargando la vida de la batería a más de un año.

Ciclo de vida: El nodo se enciende, configura sus periféricos y establece contacto con el Gateway. Una vez confirmada la sesión, alterna entre reporte y Deep Sleep.

2.2.2 Nodo Actuador

El Nodo Actuador es la unidad de acción del sistema. Recibe órdenes y las traduce en señales físicas que controlan los elementos del invernadero.

Capacidades:

- Controla pines de salida digital conectados a relés (bombas, válvulas, motores).
- Modos de seguridad: **ACTIVO** (ejecuta comandos) e **IDLE** (ignora cualquier orden, cortacircuito lógico ante mantenimiento).

- Implementa **retroalimentación confirmada**: tras ejecutar un comando envía automáticamente un informe PIN_REPORT de vuelta al servidor.
- Soporta ejecución de **secuencias multipaso temporizadas** sin bloquear el bucle principal.

2.2.3 Nodo Gateway

El Nodo Gateway es la unidad de interconexión. No mide ni actúa, sino que es el intérprete entre la red de radiofrecuencia local y la Raspberry Pi.

Capacidades:

- Habla simultáneamente ESP-NOW (RF) y UART (serial), actuando como convertidor de medio físico sin modificar el contenido de los mensajes.
- Mantiene una tabla de rutas: asocia el identificador numérico de cada nodo con su dirección MAC física.
- Reenvía comandos de la Raspberry Pi (formato binario por UART) al nodo destino correcto en la red ESP-NOW, sin interpretarlos.
- Escucha todos los mensajes de la red y los reenvía transparentemente hacia la Raspberry Pi por UART.

2.3 Arquitectura Interna en Capas

El firmware está organizado en cuatro capas claramente separadas por responsabilidad:

Cuadro 2.1: Capas del firmware Demeter

Nivel	Capa	Responsabilidad
4 (Alto)	Lógica de Nodo	Identidad y comportamiento específico del rol.
3	Orquestador	Máquina de estados, handshake, secuenciador.
2	Motor de Protocolo	Serialización/deserialización y enrutamiento.
1 (Bajo)	Estrategia de Transporte	Canal físico: ESP-NOW, UART o mixto.

2.3.1 El Orquestador del Sistema

El orquestador es el componente más complejo del firmware. Funciona como director de orquesta gestionando:

- **Máquina de estados**: BOOT → IDLE → HANDSHAKE → RUNNING → ERROR. Ninguna acción se ejecuta si el estado no lo permite.
- **Establecimiento de sesión** inspirado en TCP de tres pasos (SYN → SYN-ACK → ACK) con timeout de 2 segundos y hasta 3 reintentos.
- **Gestión de secuencias**: ejecución no bloqueante de listas ordenadas de pasos con temporización individual, evaluada en cada iteración del bucle.
- **Sistema de listeners (callbacks)**: permite suscribirse a eventos sin modificar la lógica central del orquestador.

2.4 El Motor de Protocolo

El Motor de Protocolo (Capa 2) actúa como el **corazón de la comunicación** del firmware. Es completamente agnóstico al canal físico subyacente: el mismo motor gestiona tramas tanto si provienen de ESP-NOW como de UART.

2.4.1 Responsabilidades del Motor

1. **Serialización:** transforma un comando de alto nivel (estructura de datos) en una trama binaria válida con cabecera, payload codificado en Little Endian y CRC.
2. **Deserialización:** analiza una secuencia de bytes entrante, valida el marcador de inicio (SYNC 0xAA), verifica el CRC y extrae el payload según el tipo de comando (CMD_ID).
3. **Enrutamiento:** trabaja con la tabla de rutas (ID numérico ↔ MAC física) para determinar el destinatario correcto de cada trama.
4. **Clasificación:** identifica el tipo de mensaje recibido y lanza el evento interno correspondiente (callback listener).

2.4.2 Pipeline de Recepción

Cuando el motor recibe bytes (desde cualquier transport), sigue este pipeline de validación:

1. Detección del byte de sincronía (0xAA).
2. Lectura de cabecera completa (5 bytes adicionales).
3. Verificación de que el destino (DST_ID) coincide con este nodo o es broadcast.
4. Lectura del payload de longitud declarada (LENGTH bytes).
5. Cálculo y comparación del CRC. Si no coincide: **trama descartada silenciosamente**.
6. Dispatch al listener registrado para el CMD_ID recibido.

2.5 Estrategias de Transporte

La Capa 1 del firmware abstrae el canal físico mediante el patrón **Estrategia** (*Strategy Pattern*): el Motor de Protocolo (Capa 2) no sabe nada sobre el canal físico subyacente. Simplemente llama a `transport.send(bytes)` y registra `transport.on_receive(callback)`.

Existen tres implementaciones de transporte:

2.5.1 Transporte ESP-NOW

ESP-NOW es el protocolo propietario de Espressif para comunicación directa entre microcontroladores ESP32 sin necesidad de punto de acceso Wi-Fi. Es la columna vertebral de la **red de campo** de Demeter.

Características en el contexto de Demeter:

- **Alcance:** hasta 200–400 metros en campo abierto con antenas externas.
- **Latencia:** sub-milisegundo en condiciones normales.
- **MTU:** máximo 250 bytes por trama; suficiente para cualquier trama Demeter típica.

- **Sin confirmación:** ESP-NOW no garantiza entrega; la Capa de Aplicación aporta el ACK mediante el protocolo Demeter.
- **Direccionamiento:** por dirección MAC de 6 bytes, mapeada a un ID numérico en la tabla de rutas del Gateway.

Los nodos Sensor y Actuador usan ESP-NOW como canal **exclusivo**. El Nodo Gateway usa ESP-NOW como canal de *campo* y UART como canal de *troncal*.

2.5.2 Transporte UART (Serial)

UART es la conexión física cableada entre el **Nodo Gateway ESP32** y la **Raspberry Pi**, mediante los pines TX/RX a 115200 baudios.

Características en el contexto de Demeter:

- **Fiabilidad:** al ser cableado, prácticamente sin pérdida de tramas.
- **Velocidad:** 115200 bps → capacidad de ≈ 11.500 bytes por segundo, vastamente superior a la carga de telemetría esperada.
- **Transparencia:** el Gateway reenvía las tramas binarias íntegras sin interpretarlas, actuando como pasarela pura.
- **Gestión en ambos extremos:** el ESP32 escribe/lee sobre el UART hardware; la Raspberry Pi usa `pyserial-asyncio` de forma no bloqueante.

2.5.3 Transporte Mixto (Gateway)

El Nodo Gateway implementa **dos transportes simultáneos**: escucha en ESP-NOW y retransmite los bytes recibidos por UART hacia la Raspberry, y viceversa. Esta operación de *relay* es completamente transparente: el Motor de Protocolo de la Raspberry ve las tramas como si provinieran directamente de los nodos de campo.

Decisión de Diseño

La modularidad de la capa de transporte permite añadir nuevos medios físicos (LoRa, BLE, CAN Bus) sin modificar el Motor de Protocolo ni la Lógica de Nodo. Solo se implementa la interfaz `TransportStrategy` y se registra en el orquestador.

Capítulo 3

Servicio de Borde: Raspberry Pi

3.1 Misión y Posición en la Arquitectura del Sistema

La Raspberry Pi ocupa el eslabón más crítico de toda la arquitectura Demeter: actúa como **puerta de enlace entre el mundo físico y el mundo digital**. Sin ella, no existe ningún camino técnico para que la red de sensores y actuadores ESP32 —que operan bajo un protocolo binario propietario de radiofrecuencia— se comuniquen con el servidor central en la nube.

Su responsabilidad puede enunciarse de forma simple: *escuchar la red de campo, comprender sus mensajes, y retransliterar esa información al lenguaje del servidor*; y a la inversa, *recibir órdenes del servidor y traducirlas en señales tangibles para los nodos físicos*.

Esta posición intermedia, también llamada **Edge Gateway** o Capa de Borde, es especialmente relevante en arquitecturas IoT industriales porque evita que la lógica de protocolo de campo contamine el servidor, y que el servidor requiera acceso directo al hardware.

Cuadro 3.1: Tecnologías del servicio de borde

Componente	Tecnología y función
Lenguaje UART	Python 3.9+ con modelo de concurrencia asyncio pyserial-asyncio : acceso no bloqueante al puerto serie
Red	WebSocket sobre TLS (wss://) via cable Ethernet
Despliegue	Contenedor Docker con mapeo de dispositivo físico al host
Protocolo IoT	Protocolo Demeter (propietario, binario, CRC-16)

3.2 Contenedorización y Acceso a Recursos del Sistema

El servicio Python se ejecuta íntegramente dentro de un contenedor **Docker**, lo que garantiza un entorno reproducible y aislado del sistema operativo subyacente de la Raspberry Pi (Raspberry Pi OS).

El despliegue tiene tres características relevantes:

1. **Mapeo del puerto serie**: El dispositivo físico `/dev/serial0` del host es inyectado directamente dentro del contenedor mediante la directiva de mapeo de dispositivos de Docker. Esto permite que el proceso Python en el contenedor lea y escriba en el cable UART como si estuviera en la máquina física, sin pérdida de rendimiento ni abstracción adicional.
2. **Conectividad de red**: La Raspberry Pi utiliza exclusivamente **cable Ethernet** para la conexión a internet. Esta decisión es deliberada: elimina la latencia y la inestabilidad del Wi-Fi,

especialmente relevante en entornos de invernadero con mucha interferencia de RF. El contenedor hereda la red del host de forma directa.

3. **Variables de entorno:** La URL del servidor (`WS_URL`), la velocidad del puerto serie (`BAUD_RATE`), y los credenciales de la red son inyectados como variables de entorno en tiempo de ejecución, permitiendo cambiar el entorno destino (desarrollo, producción) sin recompilar la imagen.

El Diagrama de Despliegue completo, mostrando la relación entre el host, el contenedor, el servidor y los nodos ESP32, se encuentra en la Sección B.4 del Anexo.

3.3 El Motor de Concurrencia: `asyncio`

El servicio no emplea hilos del sistema operativo (*threads*) para gestionar múltiples operaciones en paralelo. En su lugar, adopta el modelo de **concurrencia cooperativa** de Python mediante `asyncio`: un único hilo de ejecución gestiona múltiples tareas que voluntariamente ceden el control cuando están esperando una operación de entrada/salida.

Esto resulta especialmente eficiente para un servicio de este tipo, donde el 99% del tiempo de espera es I/O (esperar bytes del puerto serie, o esperar mensajes de la red).

Las cuatro tareas concurrentes que el orquestador lanza al inicio son:

Cuadro 3.2: Tareas `asyncio` del servicio Raspberry Pi

#	Tarea	Responsabilidad
1	UART RX Loop	Lee bytes del puerto serie de forma continua y no bloqueante.
2	UART TX Queue	Consume de una cola FIFO interna y escribe tramas al puerto.
3	WebSocket Client	Mantiene la conexión persistente con el servidor y envía telemetría.
4	WS Message Listener	Recibe y despacha mensajes entrantes (comandos) desde el servidor.

El flujo completo del bucle de eventos y cómo se interconectan estas cuatro tareas a través de colas asíncronas puede consultarse en la Sección B.5 del Anexo.

3.4 El Módulo del Protocolo Demeter en la Raspberry Pi

La Raspberry Pi no se limita a retransmitir bytes en bruto: **comprende el Protocolo Demeter**. Para ello, el servicio incorpora la misma lógica de serialización y deserialización descrita en el Capítulo 1, pero implementada en Python.

El módulo *UART Processor* actúa como la frontera de decodificación. Cada byte recibido pasa por un *pipeline* de validación:

1. Detección del byte de sincronía de inicio (`0xAA`).
2. Lectura de la cabecera: flags, nodo origen, nodo destino, tipo de comando y longitud.
3. Verificación del *Checksum* CRC-16 sobre el conjunto completo de la trama.
4. Desglose del *payload* según el tipo de comando reconocido.
5. Disparo del evento interno correspondiente hacia las capas superiores.

Si cualquiera de estos pasos falla, la trama es descartada silenciosamente y no se propaga ningún efecto, siguiendo la misma filosofía de fallo silente definida en el protocolo.

La información extraída de tramas válidas se clasifica según su tipo:

Cuadro 3.3: Tipos de trama reconocidos por el servicio Raspberry Pi

Tipo	Nombre	Acción desencadenada
TH_RPT	Sensor Report	Publica datos de temperatura, humedad y suelo al servidor.
SYN	Handshake Step 1	Responde con SYN-ACK y registra el nodo en el Device Manager.
ACK	Confirmación	Marca la sesión como establecida para el nodo origen.
CMD_ACK	Confirm. Actuador	Actualiza el estado del pin/actuador en la caché local.
NACK	Error de Nodo	Notifica al servidor que el comando fue rechazado por el nodo.

3.5 Comunicación con el Backend: El Canal WebSocket

La Raspberry Pi se conecta al servidor central usando un **WebSocket permanente y cifrado** sobre TLS (**wss://**), identificándose con el identificador reservado **raspberry_gateway**. Este identificador es reconocido por el servidor como la fuente de verdad de la red de campo, y tiene privilegios especiales.

Esta decisión de diseño tiene implicaciones significativas:

Decisión de Diseño

Separación de canales: El canal WebSocket es exclusivo para comunicaciones máquina a máquina (Raspberry ↔ Backend). Los clientes del navegador (Frontend) usan el mismo protocolo WebSocket, pero mediante un identificador diferente y únicamente en modo *recepción*. Los comandos desde el navegador se envían como peticiones HTTP normales; el servidor los convierte en mensajes WebSocket hacia la Raspberry internamente. Esto simplifica el frontend al eliminar la lógica de reconexión y autenticación de sockets en el navegador.

La conexión WebSocket es bidireccional:

- **Dirección ascendente (RPI → Servidor):** La Raspberry publica eventos JSON para cada trama de telemetría recibida, confirmaciones de actuadores y nuevos nodos.
- **Dirección descendente (Servidor → RPI):** El servidor envía comandos JSON (**set_gpio**, **exec_sequence**, **ping**) que el Command Dispatcher traduce a tramas binarias.

El flujo ascendente se ilustra en la Sección B.2 del Anexo. El flujo descendente se ilustra en la Sección B.3 del Anexo.

3.6 El Command Dispatcher: Adaptador JSON ↔ Binario

El componente más delicado de la Raspberry Pi para el flujo de comandos es el **Command Dispatcher**. Su rol es exactamente opuesto al del *UART Processor*: mientras este último convierte tramas binarias en estructuras de datos de alto nivel, el Dispatcher hace el camino inverso.

Cuando el servidor envía un comando JSON como:

Listing 3.1: Ejemplo de comando JSON recibido por la Raspberry Pi

```
{
  "type": "set_gpio",
  "target_id": 3,
  "pin": 4,
  "value": 1,
  "duration_ms": 15000
}
```

El Dispatcher realiza los siguientes pasos:

1. **Validación de esquema:** Verifica que el JSON contenga todos los campos requeridos para el tipo de comando indicado (**type**).
2. **Resolución de destino:** Consulta al Device Manager para verificar que el nodo **target_id** está registrado y activo.
3. **Empaquetación del payload:** Serializa los parámetros del comando (**pin**, **value**, **duration_ms**) siguiendo el formato binario del Protocolo Demeter.
4. **Construcción de la cabecera:** Prepende el byte de sincronía, los identificadores de origen y destino, el código de comando y la longitud.
5. **Cálculo del CRC-16:** Genera el checksum de todo el paquete.
6. **Encola en la TX Queue:** Inserta los bytes finales en la cola de transmisión del módulo UART, desde donde la tarea *TX Loop* los enviará al puerto serie en la siguiente iteración del bucle de eventos.

3.7 Device Manager: La Caché Local de Topología

Para no depender de una consulta al servidor en cada operación, la Raspberry Pi mantiene **en memoria una tabla de estado local** de todos los nodos conocidos en la red.

El *Device Manager* es el módulo responsable de esta caché. Registra:

- **Identidad del nodo:** identificador numérico y dirección MAC.
- **Instante de conexión:** timestamp del último mensaje recibido (*last_seen*).
- **Nivel de batería:** último valor reportado de tensión de la batería.
- **Estado de pines:** último estado conocido (ON/OFF) de cada salida de actuación.

Esta información permite al Dispatcher verificar si un nodo está activo antes de enviar un comando, y al *UART Processor* determinar si una trama SYN proviene de un nodo previamente conocido o de uno nuevo que requiere registro. La persistencia de esta tabla en disco (archivo **devices.json**) garantiza que tras un reinicio del contenedor, el servicio no pierde la memoria de los nodos con los que ya ha establecido sesión.

Capítulo 4

Servidor Central: Backend en Google Cloud

4.1 Visión General del Servidor

El **servidor central** de Demeter es el núcleo digital del sistema: el punto de convergencia donde la telemetría de campo adquiere persistencia, donde los experimentos científicos cobran forma, y desde donde los investigadores pueden actuar sobre el entorno físico del invernadero. Está desplegado en una **máquina virtual de Google Cloud Platform** (GCP), con acceso público protegido por TLS y orientado a múltiples clientes simultáneos.

El software del servidor está organizado en un conjunto de **contenedores Docker independientes** que colaboran a través de una red interna privada, sin puertos expuestos entre ellos al exterior, siguiendo el principio de mínimo acceso. Un **Nginx** actúa como proxy inverso único, terminando TLS y dirigiendo el tráfico al servicio correcto.

La arquitectura completa de contenedores, volúmenes y comunicaciones se muestra en la Sección C.1 del Anexo.

4.2 Arquitectura de Contenedores Docker

La pila de producción del servidor se compone de **seis contenedores** que conviven en una red privada interna denominada **demeter-net**:

Cuadro 4.1: Contenedores del sistema Demeter en producción

Contenedor	Imagen base	Escala	Función
demeter-frontend	Nginx + React build	1 instancia	SPA distribuida al navegador.
demeter-api	Python (FastAPI)	1 instancia	API REST + WebSocket + Lógica.
demeter-math-worker	Python (misma base)	Escalable	Worker de cálculos pesados.
demeter-sequencer	Python (misma base)	1 instancia	Motor de secuencias temporizadas.
demeter-db	PostgreSQL 15	1 instancia	Persistencia relacional.
demeter-redis	Redis Alpine	1 instancia	Broker, caché y cola de tareas.

Volúmenes persistentes:

- **postgres_data**: datos de la base de datos PostgreSQL.
- **shared_data**: archivos intermedios generados por el worker y accesibles desde la API (`/app/shared`).
- **export_data**: archivos de exportación de datos históricos (`/app/exports`).

Variables de entorno: cada contenedor recibe su configuración mediante un fichero `.env`, evitando credenciales en el repositorio de código. La URL de Redis, PostgreSQL, los secretos JWT y las claves de API son inyectadas en tiempo de ejecución.

4.3 Redis: Bus de Mensajes y Caché

Redis desempeña el rol más transversal del sistema: no es únicamente una caché de velocidad, sino el **bus de mensajes central** que cohesiona los distintos procesos Python que corren en contenedores independientes. Opera con dos bases de datos lógicas:

Cuadro 4.2: Bases de datos Redis y su propósito

DB	URL	Datos
DB/0	redis://...6379/0	Caché de aplicación y Pub/Sub
DB/1	redis://...6379/1	Cola de tareas RQ (<i>demeter_tasks</i>)

4.3.1 Las Cuatro Categorías de Datos en Redis

- Estado de Dispositivo** (`device:{id}:state`)
Almacena el estado ON/OFF *confirmado* de cada pin de actuador, tras recibir la confirmación de ejecución del nodo (CMD_ACK). Persiste indefinidamente hasta ser sobrescrito.
- Caché de Intención** (`demeter:cache:device:{id}`, TTL 60 s)
Cuando el usuario pulsa un botón, el estado *previsto* se almacena inmediatamente con un TTL de 60 segundos. El dashboard refleja este estado optimista antes de recibir confirmación del hardware, haciendo la interfaz percibida como instantánea.
- Telemetría** (`sensor:{id}:telemetry`, TTL 60 s)
Cada nueva trama de sensor que llega desde la Raspberry Pi es almacenada en Redis con expiración de un minuto. Endpoints de tiempo real la consultan directamente para responder en microsegundos, sin tocar PostgreSQL.
- Lote de Actividad** (`demeter:activity:batch`, cola FIFO)
Cada acción del usuario (botón pulsado, secuencia ejecutada) se encola en Redis en lugar de escribirse directamente en PostgreSQL. Una tarea en background (*Batch Flusher*) los vuelca periódicamente en la tabla `ActivityLog` de forma agrupada, evitando una avalancha de escrituras en la base de datos ante interacciones rápidas.

4.3.2 Canales Pub/Sub

Redis Pub/Sub es el mecanismo que permite a procesos en diferentes contenedores comunicarse sin acoplamiento directo:

Cuadro 4.3: Canales Pub/Sub y sus productores/consumidores

Canal	Publicador	Suscriptor
<code>demeter:commands</code>	POST /api/command	WS Dispatcher (asyncio task)
<code>iot_events</code>	RedisManager	Frontend (retransmisión WS broadcast)

El flujo completo de un comando desde el navegador hasta el ESP32, pasando por Redis y la Raspberry Pi, se muestra en la Sección C.2 del Anexo.

4.4 PostgreSQL: Persistencia y Esquema de Datos

PostgreSQL almacena toda la información de largo plazo del sistema. El esquema evoluciona mediante **Alembic**, el gestor de migraciones de SQLAlchemy, garantizando que cualquier cambio en el modelo de datos sea reproducible y trazable en todos los entornos.

Las tablas principales son:

Cuadro 4.4: Tablas principales de PostgreSQL

Tabla	Datos almacenados
users	Credenciales de acceso, roles (admin/viewer) y estado activo.
plants	Registro botánico de plantas: especie, variedad, fecha de siembra, estado vital, metadatos científicos, nodo asignado.
experiments	Agrupación de plantas en experimentos de investigación.
experimento_planta_link	Tabla intermedia N:M entre plantas y experimentos.
telemetry_th	Serie temporal de mediciones de temperatura y humedad por nodo.
activity_log	Registro de acciones (comandos enviados, interacciones de usuario).
sequences	Secuencias de pasos de riego configuradas por los usuarios.

El diagrama entidad-relación completo puede consultarse en la Sección ?? del Anexo.

4.5 El Worker Matemático

El **Math Worker** (`demeter-math-worker`) es un proceso Python independiente que escucha la cola de tareas `demeter_tasks` en Redis (DB/1) usando la librería **RQ** (*Redis Queue*).

Su función es absorber los cálculos *pesados y no urgentes* que no deben bloquear la API principal. Las tareas que ejecuta incluyen:

- **Análisis científico por experimento:** cálculo de estadísticos de las series temporales de temperatura y humedad asociadas a un experimento (media, varianza, tendencias, correlación entre nodos).
- **Generación de informes:** empaquetamiento de los análisis en archivos de resultados exportables al volumen compartido `/app/shared`.
- **Nightly Batch:** tareas programadas de limpieza, compactación de telemetría histórica y recálculo de estadísticos agregados.

El proceso de encolar una tarea sigue este patrón: el cliente llama a `POST /api/analysis/generate`, el backend encola la tarea en Redis y devuelve un `task_id`. El cliente puede consultar `GET /api/analysis/status/{task_id}` para conocer el progreso y, una vez completada, descarga el resultado de `GET /api/analysis/download/{filename}`.

4.6 El Secuenciador

El **Secuenciador** (`demeter-sequencer`) es un proceso Python independiente con un único propósito: ejecutar secuencias de pasos de riego multi-acción de forma temporizada, sin bloquear la API ni consumir recursos del worker matemático.

Una secuencia puede definir, por ejemplo: “Abrir válvula A durante 10 segundos, esperar 2 segundos, abrir bomba B durante 30 segundos, cerrar todo”.

El Secuenciador traduce cada paso temporizado en un comando individual y lo lanza al canal Redis `demeter:commands`, que el Dispatcher convierte en una trama que llega al actuador físico. Entre paso y paso, duerme el tiempo de espera definido sin bloquear ningún otro proceso del sistema.

4.7 El Backend FastAPI

La **API principal** (`demeter-api`) es el componente central y más complejo del servidor. Implementada con el framework **FastAPI** sobre **asyncio**, combina en un único proceso el servidor HTTP REST, el servidor WebSocket, y dos tareas en background permanentes:

- **Redis Command Listener**: escucha el canal `demeter:commands` y retransmite comandos a la Raspberry Pi conectada por WebSocket.
- **Activity Batch Flusher**: cada 30 segundos vuelca el lote de eventos de actividad de Redis a PostgreSQL.

Todas las rutas que acceden a la base de datos lo hacen de forma **completamente asíncrona** mediante **AsyncSession** de SQLAlchemy, garantizando que las consultas lentas no bloqueen otras peticiones en curso.

4.8 El Frontend React

El **frontend** es una *Single Page Application* (SPA) construida en React, servida como ficheros estáticos a través de Nginx. Una vez cargada en el navegador, se comunica directamente con la API del backend mediante peticiones HTTPS y WebSocket. Ofrece las siguientes funcionalidades:

Cuadro 4.5: Módulos del frontend React

Módulo	Funcionalidades
Dashboard en tiempo real	Visualización de temperatura, humedad y suelo con gráficos actualizados vía WebSocket.
Control Manual	Botones para activar/desactivar pines de actuadores individualmente.
Planificador de Secuencias	Creación, guardado y ejecución de secuencias de riego multi-paso.
LIMS	Registro y gestión de plantas, asignación a experimentos, consulta de telemetría por planta.
SDK / Exportación	Descarga de datos en formatos académicos, acceso programático para análisis externos.

4.9 Documentación de la API REST

El servidor expone una API REST bajo el prefijo `/api`, con documentación interactiva generada automáticamente por FastAPI en `/api/docs`. A continuación se detallan todos los *routers* y sus endpoints más relevantes.

4.9.1 Sistema (`/api/system`)

Método	Ruta	Descripción
GET	/api/system/health	Comprueba el estado del servidor y la conexión a Redis y PostgreSQL. Devuelve un JSON de estado.

4.9.2 Autenticación (/api/auth)

Método	Ruta	Descripción
POST	/api/auth/login	Recibe usuario y contraseña. Devuelve un JWT Bearer Token con el rol del usuario y expiración configurable.
POST	/api/auth/register	Registra un nuevo usuario (admin). Almacena la contraseña hasheada con bcrypt.
GET	/api/auth/me	Devuelve el perfil del usuario activo autenticado mediante el token.

4.9.3 Comandos (/api/command)

Método	Ruta	Descripción
POST	/api/command	Envía un comando al hardware. El campo <code>type</code> discrimina el subtipo (<code>set_gpio</code> , <code>set_pwm</code> , <code>exec_sequence</code> , <code>ping</code> , <code>get_sensors</code>). El comando es validado con Pydantic y publicado en Redis. Devuelve 202 Accepted.

4.9.4 Descubrimiento (/api/discovery)

Método	Ruta	Descripción
GET	/api/discovery/nodes	Lista todos los nodos activos conocidos por el sistema con su último estado.
GET	/api/discovery/nodes/{id}	Estado detallado y telemetría en caché de un nodo específico.

4.9.5 Historial de Telemetría (/api/history)

Método	Ruta	Descripción
GET	/api/history/node/{id}	Devuelve hasta <code>days</code> días de telemetría histórica de temperatura y humedad para el nodo indicado. Requiere autenticación JWT.

4.9.6 LIMS — Plantas y Experimentos (/api/lims)

Método	Ruta	Descripción
GET	/api/lims/plantas	Lista todas las plantas registradas.
POST	/api/lims/plantas	Registra una nueva planta con metadatos botánicos completos.
GET	/api/lims/plantas/{id}	Detalle de una planta con sus experimentos asociados.
PATCH	/api/lims/plantas/{id}	Actualiza parcialmente los datos de una planta.
GET	/api/lims/plantas/{id}/telemetry	Telemetría de 6 meses del nodo asignado a la planta, directo de Redis.
GET	/api/lims/experimentos	Lista todos los experimentos vigentes.
POST	/api/lims/experimentos/generar	Crea un experimento y asigna plantas existentes.

4.9.7 Análisis Científico (/api/analysis)

Método	Ruta	Descripción
POST	/api/analysis/generate	Encola un análisis estadístico completo de un experimento en el Math Worker (RQ). Devuelve el <code>task_id</code> .
GET	/api/analysis/status/{id}	Consulta el estado de la tarea encolada: <i>queued</i> , <i>started</i> , <i>finished</i> o <i>failed</i> .
GET	/api/analysis/download/{f}	Descarga el archivo de análisis generado (ZIP) desde el volumen compartido.

4.9.8 SDK Científico (/api/sdk)

Método	Ruta	Descripción
GET	/api/sdk/mediciones/{exp_id}	Devuelve la telemetría cruda en JSON de todas las plantas del experimento. Protegido por X-API-Key. Prioriza caché Redis sobre consulta directa a PostgreSQL.

4.9.9 Exportación (/api/export)

Método	Ruta	Descripción
GET	/api/export/{formato}	Exporta los datos históricos en formatos CSV o XLSX para análisis externo. Se genera y devuelve como descarga directa.

4.10 El Canal WebSocket

El servidor expone el endpoint `/ws/{client_id}` como punto de entrada único para todas las conexiones WebSocket. El comportamiento varía según el identificador con el que se conecta el cliente:

Cuadro 4.6: Diferencia de comportamiento por tipo de cliente WebSocket

client_id	Tipo	Dirección	Mensajes
raspberrypi_gateway	Máquina	Bidireccional	Recibe JSON de comandos, envía JSON de telemetría.
Cualquier otro	Frontend/Monitor	Solo recepción	Solo recibe push de telemetría. Los intentos de enviar comandos son rechazados con un mensaje informativo.

Esta separación es una decisión de diseño deliberada: el canal WebSocket queda reservado para el flujo de datos en tiempo real (telemetría), mientras los comandos circulan por HTTP POST. Esto simplifica el frontend (sin lógica de reconexión de socket) y hace los comandos trivialmente verificables con herramientas estándar como `curl` o la documentación Swagger de la API.

Capítulo 5

SDK Científico de Python — `demeter_sdk`

5.1 Propósito y Filosofía

El **Demeter SDK** es la capa de acceso programático al sistema para investigadores y analistas. Nacido de la necesidad de combinar los datos de telemetría del invernadero con algoritmos agronómicos reconocidos, el SDK resuelve el ciclo completo de análisis en *una sola llamada*:

- Conectarse a la API del servidor con autenticación por clave.
- Descargar la telemetría de un experimento con caché automática.
- Limpiar y normalizar los datos (ETL).
- Calcular indicadores agronómicos derivados (VPD, GDD, ET_0 , ...).
- Exportar los resultados a CSV o Excel estructurado.
- Generar figuras de publicación científica con un único comando.

Decisión de Diseño

Filosofía de uso: el investigador no necesita conocer la arquitectura interna del sistema, ni Redis, ni WebSocket, ni la API directamente. Basta con instanciar `DemeterClient`, pasar el ID del experimento y el SDK hace el resto. El objetivo es que un científico con conocimientos básicos de Python pueda obtener un informe completo en menos de diez líneas de código.

5.2 Arquitectura Modular

El SDK está organizado en seis módulos con responsabilidades estrictamente separadas. La clase `DemeterClient` actúa como orquestador público que los une:

Cuadro 5.1: Módulos del Demeter SDK

Módulo	Rol	Clase / Función principal
<code>client.py</code>	Orquestador	<code>DemeterClient</code>
<code>fetcher.py</code>	Obtención de datos	<code>fetch_experiment()</code> , <code>fetch_plant()</code>
<code>transform.py</code>	ETL (limpieza)	<code>pipeline()</code> , <code>resample()</code>
<code>science.py</code>	Cálculos agronómicos	<code>enrich()</code> , funciones individuales
<code>viz.py</code>	Visualización científica	<code>plot_timeseries()</code> , <code>plot_vpd()</code> , ...
<code>export.py</code>	Exportación de resultados	<code>to_csv()</code> , <code>to_excel()</code>

5.3 Instalación y Configuración

El SDK se instala como un paquete Python local desde la carpeta del proyecto:

Listing 5.1: Instalación del SDK en modo editable

```
pip install -e ./SDK/
```

La autenticación se realiza mediante una **clave API** (X-API-Key) generada por el sistema. Puede pasarse directamente al constructor o mediante la variable de entorno **DEMETER_API_KEY**:

Listing 5.2: Inicialización del cliente

```
from demeter_sdk import DemeterClient

# Opción 1: API key explícita
client = DemeterClient(
    api_key = "dk_live_XXXXXXXXXXXX",
    base_url = "https://demeter.tuservidor.com",
    cache_ttl = 3600, # Caché de 1 hora
)

# Opción 2: variable de entorno DEMETER_API_KEY
client = DemeterClient(base_url="https://demeter.tuservidor.com")
```

5.4 Obtención de Datos: Módulo Fetcher

El fetcher implementa una caché transparente en formato **Apache Parquet** para evitar descargar los mismos datos repetidamente durante análisis iterativos. El comportamiento es:

- Si existe un fichero Parquet en `~/.demeter_cache/` con antigüedad inferior al TTL configurado (defecto: 1 h), se devuelve sin llamar a la API.
- Si el fichero es más antiguo o no existe, se descarga desde la API y se guarda.
- `force_refresh=True` omite la caché siempre.

Listing 5.3: Descarga de datos con caché automática

```
# Datos crudos de un experimento (últimos 30 días)
raw = client.get_raw_data(experimento_id=1, dias=30)

# Datos crudos de una planta individual (sirve el endpoint Redis)
raw_planta = client.get_plant_data(plant_id=7)

# Forzar recarga desde la API
raw_fresh = client.get_raw_data(experimento_id=1, force_refresh=True)
```

5.5 Pipeline ETL: Módulo Transform

Antes de calcular cualquier indicador, los datos crudos pasan por un pipeline de limpieza en cuatro etapas:

Cuadro 5.2: Etapas del pipeline ETL

Paso	Función	Qué hace
1	<code>to_dataframe()</code>	Convierte los JSON crudos a un DataFrame tipado. Parsea timestamps ISO-8601, coerce numérico de temperatura/humedad, ordena cronológicamente.
2	<code>clean()</code>	Elimina filas duplicadas exactas y mediciones con ambos sensores a NaN (irrecuperables).
3	<code>fill_gaps()</code>	Interpolación lineal de huecos puntuales de sensor (desconexiones breves). Soporta también método 'cubic' o 'time'.
4	<code>resample()</code>	Opcional. Agrega la serie temporal a menor frecuencia (e.g. '1h', '1D') por nodo.

Listing 5.4: Uso del pipeline ETL

```
# Pipeline completo con resampleado horario
df = client.get_clean_data(
    experimento_id=1,
    dias=30,
    resample_rule="1h", # Opcional
)
# Resultado: DataFrame con columnas [timestamp, temperature, humidity, node_id]
```

5.6 Cálculos Agronómicos: Módulo Science

El módulo `science` implementa **todas las fórmulas** agronómicas de forma vectorizada sobre NumPy/Pandas, sin bucles Python. Todas las funciones admiten arrays o `pd.Series` y devuelven `np.ndarray`.

5.6.1 Termodinámica del Aire Húmedo

Presión de Vapor de Saturación (e_s)

Calculada mediante la fórmula de **Magnus-Tetens**, el estándar de la meteorología agrícola para el rango de temperaturas habitual en invernadero (-10 a 50°C):

$$e_s = 0,6108 \cdot e^{\frac{17,27 \cdot T}{T + 237,3}}$$

In: T — Temperatura de bulbo seco ($^\circ\text{C}$).

Out: e_s — Presión de vapor de saturación (kPa).

Déficit de Presión de Vapor (VPD)

El VPD es el indicador más útil para el manejo del invernadero. Mide la diferencia entre la cantidad de agua que el aire *podría* contener (e_s) y la que *realmente* contiene ($e_a = e_s \cdot HR/100$):

$$VPD = \max(0, e_s - e_a)$$

In: e_s — Presión de vapor de saturación (kPa).

In: e_a — Presión de vapor actual (kPa). Se calcula como $e_s \cdot HR/100$.

Out: VPD — Déficit de presión de vapor (kPa).

La tabla siguiente define las zonas de riesgo para cultivos de invernadero típicos:

Cuadro 5.3: Zonas de riesgo del VPD para cultivos de invernadero

VPD (kPa)	Zona	Implicación agronómica
$< 0,4$	Riesgo fúngico	Exceso de humedad; condensación en hoja probable.
$0,4 - 1,0$	Óptimo	Transpiración activa, absorción de nutrientes eficiente.
$1,0 - 1,5$	Estrés moderado	Inicio de cierre estomático; reducir temperatura.
$> 1,5$	Estrés severo	Cierre estomático; parar riego por goteo.

Punto de Rocío (T_d)

Temperatura a la que el aire se satura si se enfría isohúumamente. Cuando la temperatura de una superficie (e.g. hoja) cae por debajo de T_d , hay condensación. Se calcula invirtiendo la fórmula de Magnus:

$$T_d = \frac{237,3 \cdot \ln(e_a/0,6108)}{17,27 - \ln(e_a/0,6108)}$$

In: e_a — Presión de vapor actual (kPa).

Out: T_d — Temperatura de punto de rocío ($^{\circ}\text{C}$).

Temperatura de Bulbo Húmedo (T_w) — Stull 2011

Temperatura límite del enfriamiento por evaporación. Usada en sistemas de refrigeración evaporativa de invernadero:

$$T_w = T \cdot \arctan\left(0,151977 \cdot \sqrt{HR + 8,313659}\right) + \arctan(T + HR) - \arctan(HR - 1,676331) + 0,00391838 \cdot HR^{1,5} \cdot \arctan(0,023101 \cdot HR) - 4,686035$$

In: T — Temperatura de bulbo seco ($^{\circ}\text{C}$).

In: HR — Humedad relativa (%).

Out: T_w — Temperatura de bulbo húmedo ($^{\circ}\text{C}$).

Humedad Absoluta

Masa de vapor de agua por metro cúbico de aire, independiente de la presión:

$$AH = \frac{2165 \cdot e_a}{T + 273,15}$$

In: e_a — Presión de vapor actual (kPa).

In: T — Temperatura ($^{\circ}\text{C}$).

Out: AH — Humedad absoluta (g/m^3).

Entalpía Específica del Aire Húmedo — ASHRAE

Energía total (térmica + latente) del aire húmedo por kilogramo de aire seco. Clave para el dimensionado de sistemas de climatización y refrigeración:

$$r = \frac{0,622 \cdot e_a}{P - e_a}$$

$$h = 1,006 \cdot T + r \cdot (2501 + 1,86 \cdot T)$$

In: T — Temperatura de bulbo seco (°C).

In: e_a — Presión de vapor actual (kPa).

In: P — Presión atmosférica (kPa, defecto 101,325).

Out: h — Entalpía específica del aire húmedo (kJ/kg aire seco).

Out: r — Relación de mezcla o humedad específica (kg agua/kg aire seco).

Índice de Calor (Rothfusz, NWS)

Temperatura *percibida* por el ser humano (y por el cultivo) combinando temperatura y humedad. Solo significativo para $T \geq 27^\circ\text{C}$:

$$HI = -8,785 + 1,611 T + 2,339 HR - 0,1461 T \cdot HR$$

$$- 0,01231 T^2 - 0,01642 HR^2 + 0,002212 T^2 \cdot HR$$

$$+ 0,000726 T \cdot HR^2 - 0,000004 T^2 \cdot HR^2$$

In: T — Temperatura ambiental (°C).

In: HR — Humedad relativa (%).

Out: HI — Índice de calor percibido (°C).

Riesgo de Rocío en Hoja (LSD)

Índice que evalúa si la superficie de la hoja (modelada a $T_{leaf} = T_{air} - 2^\circ\text{C}$) está por debajo de la temperatura de punto de rocío. Un valor negativo indica condensación activa y riesgo de enfermedades fúngicas:

$$LSD = e_s(T_{leaf}) - e_a(T_{air}, HR)$$

In: T_{leaf} — Temperatura de la hoja (modelada como $T_{air} - 2$, °C).

In: e_s — Presión de vapor de saturación a T_{leaf} (kPa).

In: e_a — Presión de vapor actual del aire (kPa).

Out: LSD — Riesgo de rocío en hoja (*Leaf Surface Dew*, kPa).

5.6.2 Fenología y Tiempo

Grados-Día de Crecimiento (GDD)

El GDD es la unidad de acumulación de calor vegetal; establece el ritmo de desarrollo fenológico del cultivo. Se calcula como:

$$GDD = \max\left(0, \frac{T_{max} + T_{min}}{2} - T_{base}\right)$$

In: T_{max} — Temperatura máxima diaria (°C).

In: T_{min} — Temperatura mínima diaria (°C).

In: T_{base} — Temperatura umbral de crecimiento (°C, defecto 10).

Out: GDD — Grados-Día de Crecimiento (°C · día).

Horas de Frío (*Chill Hours*)

Recuento de horas en las que la temperatura se encuentra en el rango (0, 7,2] °C. Indicador clave para predecir la ruptura de dormancia en frutales de hoja caduca.

Evapotranspiración de Referencia — Hargreaves-Samani (ET_0)

Estima la demanda hídrica de referencia usando solo datos de temperatura, sin necesidad de piranómetros ni sensores de viento:

$$ET_0 = 0,0023 \cdot R_a \cdot (T_{mean} + 17,8) \cdot \sqrt{T_{max} - T_{min}}$$

In: R_a — Radiación extraterrestre (equivalente mm/día, defecto 15).

In: T_{mean} — Temperatura media diaria (°C).

In: T_{max} — Temperatura máxima diaria (°C).

In: T_{min} — Temperatura mínima diaria (°C).

Out: ET_0 — Evapotranspiración de referencia (mm/día).

Días Tras la Siembra (DAP)

Número de días enteros transcurridos desde la fecha de siembra:

$$DAP = \lfloor t_{sample} - t_{siembra} \rfloor$$

In: t_{sample} — Fecha de la muestra (timestamp).

In: $t_{siembra}$ — Fecha de siembra (timestamp).

Out: DAP — Días tras la siembra (días).

Permite contextualizar cada medición en términos del ciclo de vida del cultivo.

5.6.3 Estadística y Detección de Anomalías

Z-Score

Normalización lineal de una serie temporal. Detecta cuántas desviaciones estándar se aleja cada muestra de la media del periodo:

$$z = \frac{x - \mu}{\sigma}$$

In: x — Valor de la muestra.

In: μ — Media aritmética de la serie.

In: σ — Desviación estándar de la serie.

Out: z — Puntuación estándar (adim.).

El SDK aplica Z-Score de forma independiente a temperatura y humedad. Se considera outlier cualquier muestra con $|z| > 3$ (umbral configurable).

Índice de Riesgo Wallin (1962) — Tizón Tardío

Modelo clásico de epidemiología vegetal para estimación del riesgo de *Phytophthora infestans* (mildiu/tizón tardío). Evalúa un periodo horario y devuelve un nivel de riesgo:

Cuadro 5.4: Lógica del índice de riesgo Wallin

Condición	Nivel	Acción recomendada
Horas con $HR \geq 90\% < 10$	0 — Sin riesgo	Sin acción.
Horas ≥ 10 y $10^{\circ}\text{C} \leq T \leq 27^{\circ}\text{C}$	1 — Riesgo moderado	Revisión preventiva.
Horas ≥ 20	2 — Riesgo alto	Tratamiento fungicida.

Media Móvil 24h

Suavizado de la serie temporal por grupos de nodo, usando una ventana de 24 muestras (equivalente a 24 h cuando los datos son horarios). Útil para eliminar el ruido de alta frecuencia y visualizar tendencias.

5.7 El Helper `enrich()`: Enriquecimiento en Una Llamada

La función `science.enrich()` aplica en un solo paso todos los cálculos sobre el DataFrame limpio y devuelve un nuevo DataFrame con las columnas derivadas:

Cuadro 5.5: Columnas añadidas por `enrich()`

Columna	Unidad	Fórmula / Método
<code>vpd_kpa</code>	kPa	Magnus-Tetens
<code>dew_point_c</code>	°C	Inversión Magnus
<code>wet_bulb_c</code>	°C	Stull 2011
<code>abs_humidity_g_m3</code>	g/m ³	Ley del gas ideal
<code>enthalpy_kj_kg</code>	kJ/kg	Definición ASHRAE
<code>heat_index_c</code>	°C	Rothfusz (NWS)
<code>lsd_kpa</code>	kPa	Modelado hoja -2°C
<code>z_score_temp</code>	adim.	$(T - \mu)/\sigma$
<code>z_score_hum</code>	adim.	$(HR - \mu)/\sigma$
<code>temp_ma_24h</code>	°C	Media móvil 24 muestras
<code>hum_ma_24h</code>	%	Media móvil 24 muestras
<code>is_outlier</code>	bool	$ z_{temp} > 3,0$
<code>dap_days</code>	días	Solo si <code>fecha_siembra</code> provisto

Listing 5.5: Pipeline completo con enriquecimiento

```
# Datos limpios + todos los indicadores agronómicos
df = client.get_enriched_data(
    experimento_id = 1,
    dias = 30,
    fecha_siembra = "2025-09-01", # Activa columna dap_days
    resample_rule = "1h", # Opcional: resampleado horario
)

# Consultar el VPD medio de cada nodo
print(df.groupby("node_id")["vpd_kpa"].mean())

# Ver anomalías de temperatura
outliers = df[df["is_outlier"]]
print(f"{len(outliers)} lecturas anómalas detectadas")
```

5.8 Visualización Científica: Módulo Viz

Todos los gráficos están implementados con Matplotlib y Seaborn con estilo consistente y devuelven el objeto `Figure` para que el investigador pueda añadir anotaciones, cambiar el título o guardar con el DPI deseado.

Cuadro 5.6: Funciones de visualización disponibles

Función	Tipo	Descripción
<code>plot_timeseries()</code>	Serie temporal	Cualquier métrica frente al tiempo, coloreada por nodo, con media móvil superpuesta.
<code>plot_vpd()</code>	VPD con zonas	VPD en el tiempo con bandas de color que marcan las 4 zonas de riesgo agronómico.
<code>plot_boxplot()</code>	Diagrama de cajas	Distribución de cualquier métrica por nodo; ideal para comparar variabilidad entre plantas.
<code>plot_heatmap()</code>	Mapa de calor 2D	Intensidad de la métrica en el tiempo (eje X) por nodo (eje Y); resampleado diario por defecto.

Listing 5.6: Generación de gráficos

```
# Serie temporal del VPD con zonas de riesgo
fig = client.plot.plot_vpd(df)
fig.savefig("vpd_zonas.png", dpi=300, bbox_inches="tight")

# Boxplot de temperatura por nodo
fig2 = client.plot.plot_boxplot(df, metric="temperature")
fig2.savefig("boxplot_temperatura.png", dpi=300)

# Heatmap de humedad relativa (dia x nodo)
fig3 = client.plot.plot_heatmap(df, metric="humidity", resample_rule="1D")
fig3.savefig("heatmap_humidity.png", dpi=300)
```

5.9 Exportación Científica: Módulo Export

5.9.1 CSV

El fichero CSV se escribe en codificación **UTF-8 con BOM** (`utf-8-sig`) para asegurar la apertura correcta en Excel sin selección manual de codificación:

Listing 5.7: Exportación a CSV

```
client.export.to_csv(df_enriched, "resultados/telemetria.csv")
```

5.9.2 Excel Multi-hoja

El libro Excel generado tiene tres hojas estructuradas para la comunicación científica:

Cuadro 5.7: Estructura del libro Excel generado

Hoja	Contenido	Utilidad
Resumen	Media, mediana, desv. típica, mín., máx., P5, P95 de todas las métricas.	Vista rápida para el investigador principal y presentaciones.
Datos_Enriquecidos	Todos los registros con todas las columnas derivadas (13+ columnas).	Dataset completo para análisis estadístico externo (R, SPSS).
Datos_Crudos	Solo temperatura y humedad originales (sin derivados).	Reproducibilidad: los datos tal y como llegaron del sensor.

Listing 5.8: Exportación a Excel estructurado

```
client.export.to_excel(
    df_enriched,
    "resultados/informe.xlsx",
    df_raw = df_raw,
    experiment_name = "Ensayo Estrés Salino 2025",
)
```

5.10 Suite Completa — Un Solo Comando

Para el flujo de análisis estándar de un experimento, el método `run_full_suite()` ejecuta automáticamente: descarga, ETL, enriquecimiento, exportación CSV + Excel y los cuatro tipos de gráfico, guardándolo todo en una estructura de carpetas organizada:

Listing 5.9: Análisis completo en una sola llamada

```
ruta = client.run_full_suite(
    experimento_id = 3,
    dias = 60,
    output_dir = "./resultados",
    fecha_siembra = "2025-10-01",
    resample_rule = "1h",
)
# Resultado:
# resultados/Experimento_3/
# |- data/
# ||- enriched_telemetry.csv
# |+- report.xlsx
# +- plots/
# |- temperature_timeseries.png
# |- vpd_zones.png
# |- temperature_boxplot.png
# +- heatmap_temperature.png
print(f"Análisis guardado en: {ruta}")
```

5.11 Ejemplo de Sesión Completa con Jupyter

El siguiente extracto reproduciría una sesión típica de investigación en un Jupyter Notebook, desde la conexión hasta la interpretación de resultados:

Listing 5.10: Sesión completa en Jupyter Notebook

```
from demeter_sdk import DemeterClient
```

```
# 1. Conectar
client = DemeterClient(api_key="dk_live_XXX",
                      base_url="https://demeter.servidor.com")
assert client.ping(), "Servidor no disponible"

# 2. Obtener datos limpios y enriquecidos
df = client.get_enriched_data(
    experimento_id=2,
    dias=30,
    fecha_siembra="2025-09-15",
    resample_rule="30min",
)

# 3. Consultas rápidas
vpd_alto = df[df["vpd_kpa"] > 1.5]
print(f"Horas con estrés severo: {len(vpd_alto)}")

wallin = client.plot.plot_vpd(df)
wallin.savefig("vpd.png", dpi=300)

# 4. Calcular GDD acumulado del experimento
from demeter_sdk.science import gdd
df_diario = df.resample("1D", on="timestamp").agg(
    T_max=("temperature", "max"),
    T_min=("temperature", "min")
).reset_index()
df_diario["gdd"] = gdd(df_diario["T_max"], df_diario["T_min"])
df_diario["gdd_acum"] = df_diario["gdd"].cumsum()
print(df_diario[["timestamp", "gdd_acum"]].tail())

# 5. Exportar
client.export.to_excel(df, "informe_exp2.xlsx", experiment_name="Exp 2")
```

5.12 Referencia Rápida de Parámetros

Cuadro 5.8: Parámetros principales del DemeterClient

Parámetro	Tipo	Descripción
api_key	str	Clave de acceso a la API (o via DEMETER_API_KEY).
base_url	str	URL base del servidor (defecto: localhost:8000).
cache_dir	str	Ruta de la caché Parquet (defecto: ~/.demeter_cache).
cache_ttl	int	Tiempo de vida en segundos de los ficheros Parquet (defecto: 3600).
timeout	int	Timeout por petición HTTP en segundos (defecto: 30).
max_retries	int	Reintentos automáticos ante errores 5xx (defecto: 3).

Apéndice A

Diagramas del Protocolo Binario Dementer

Esta sección recoge los diagramas de flujo completos del protocolo de comunicación binario implementado entre los nodos ESP32 y el Gateway.

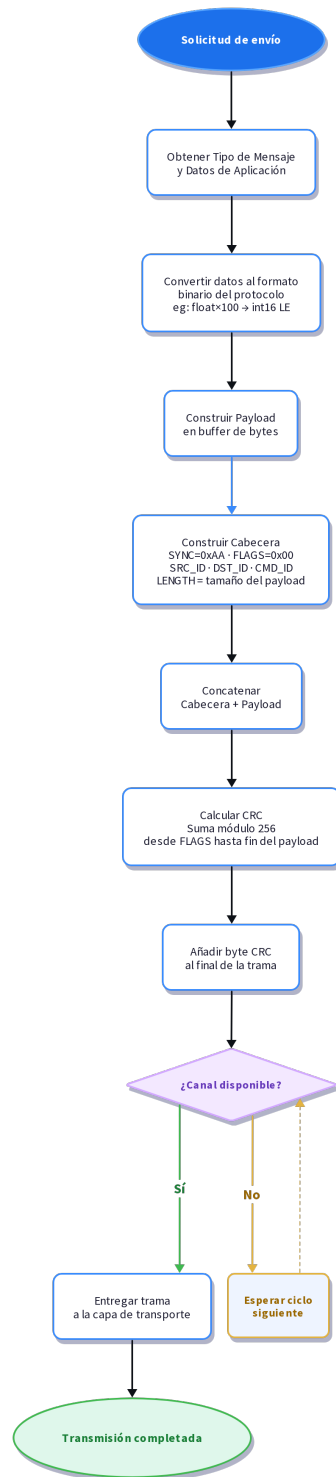


Figura A.1: Flujo de serialización de una trama binaria Demeter.

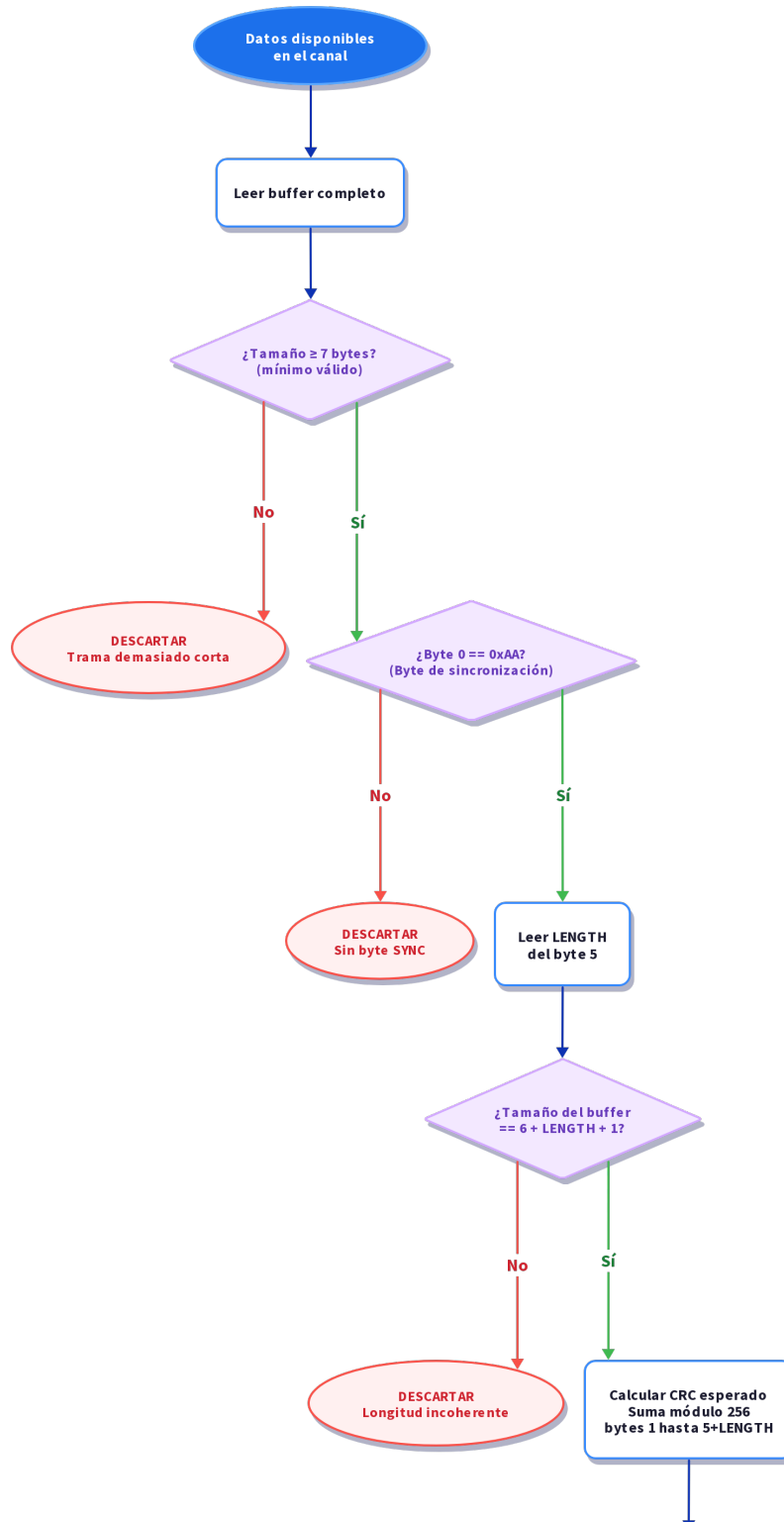


Figura A.2: Flujo de deserialización y validación de una trama recibida (parte superior).

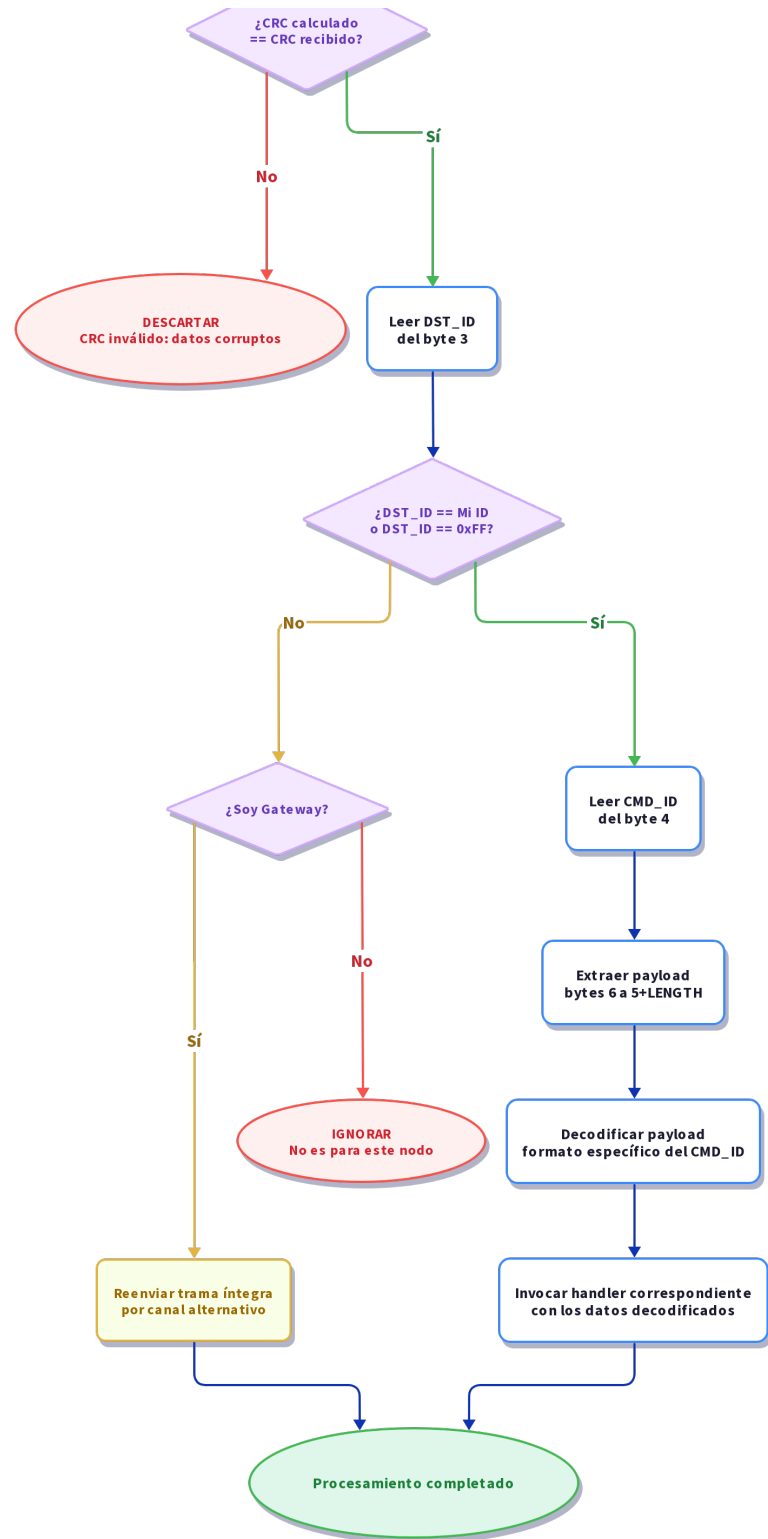


Figura A.3: Flujo de deserialización y validación de una trama recibida (parte inferior).

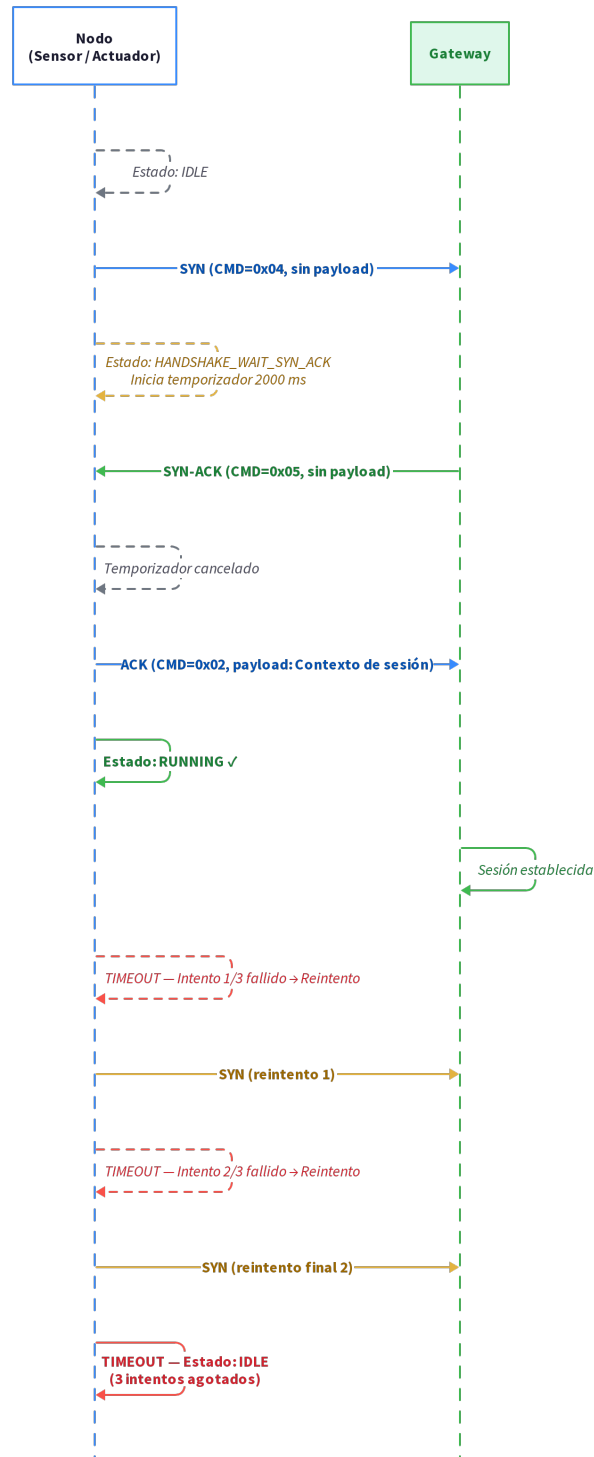


Figura A.4: Diagrama de secuencia del handshake SYN / SYN-ACK / ACK con reintentos.

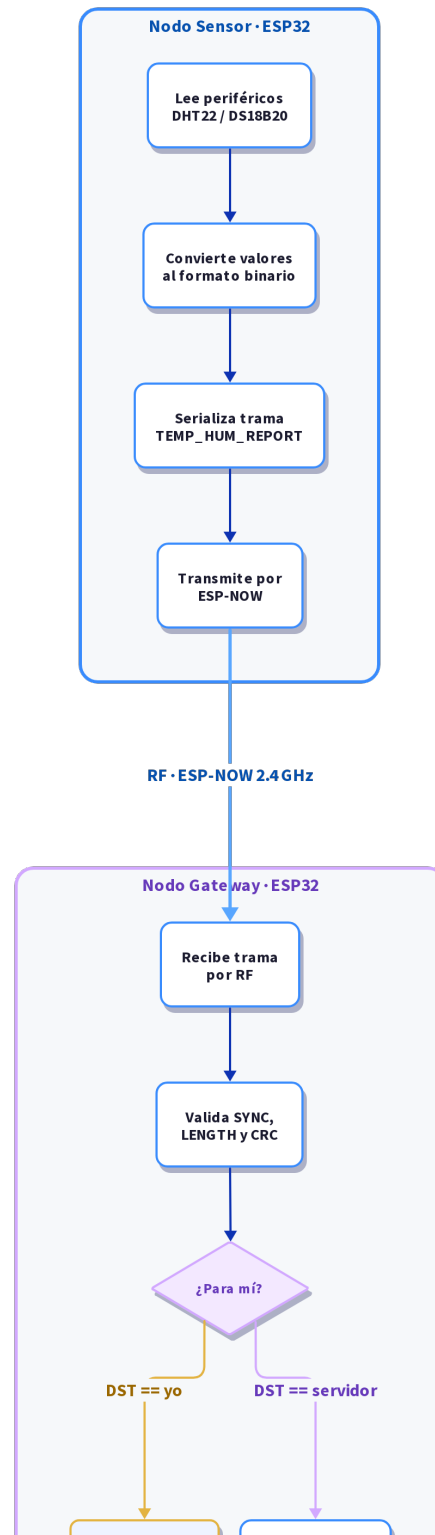


Figura A.5: Flujo end-to-end: Nodo Sensor → Gateway → Raspberry Pi → Backend (parte superior).

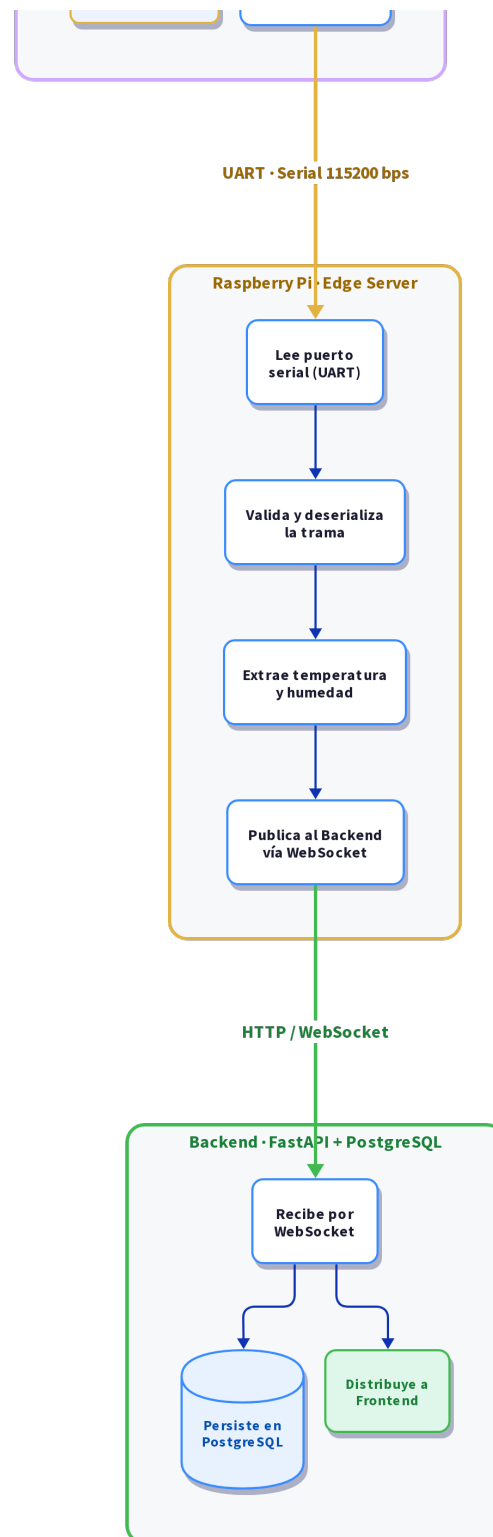


Figura A.6: Flujo end-to-end: Nodo Sensor → Gateway → Raspberry Pi → Backend (parte inferior).

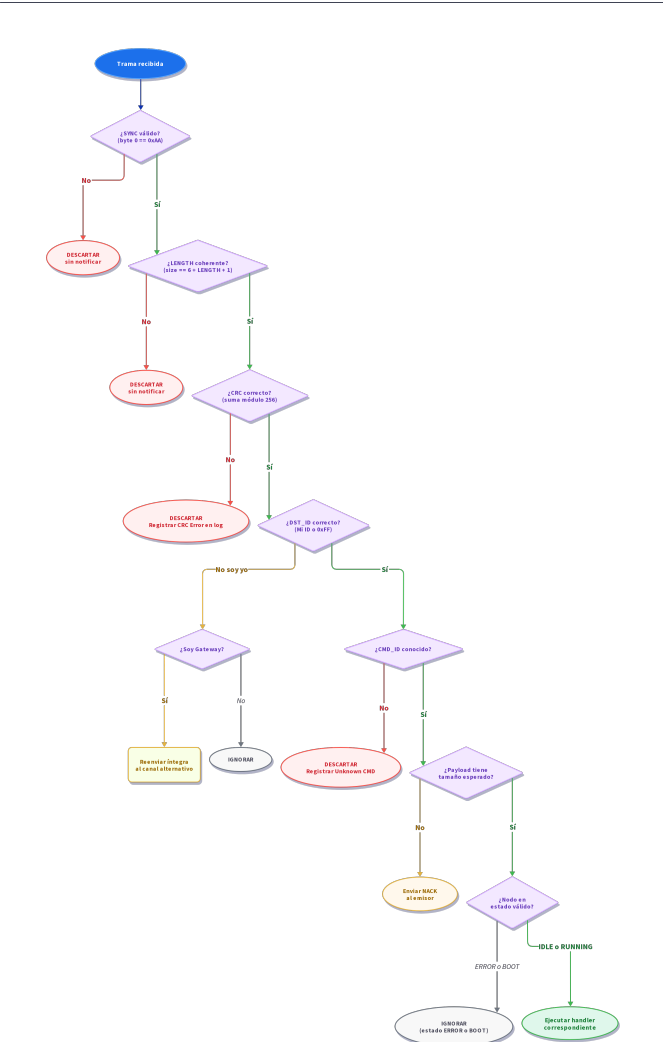


Figura A.7: Árbol de decisión para el manejo de errores en la recepción de tramas.

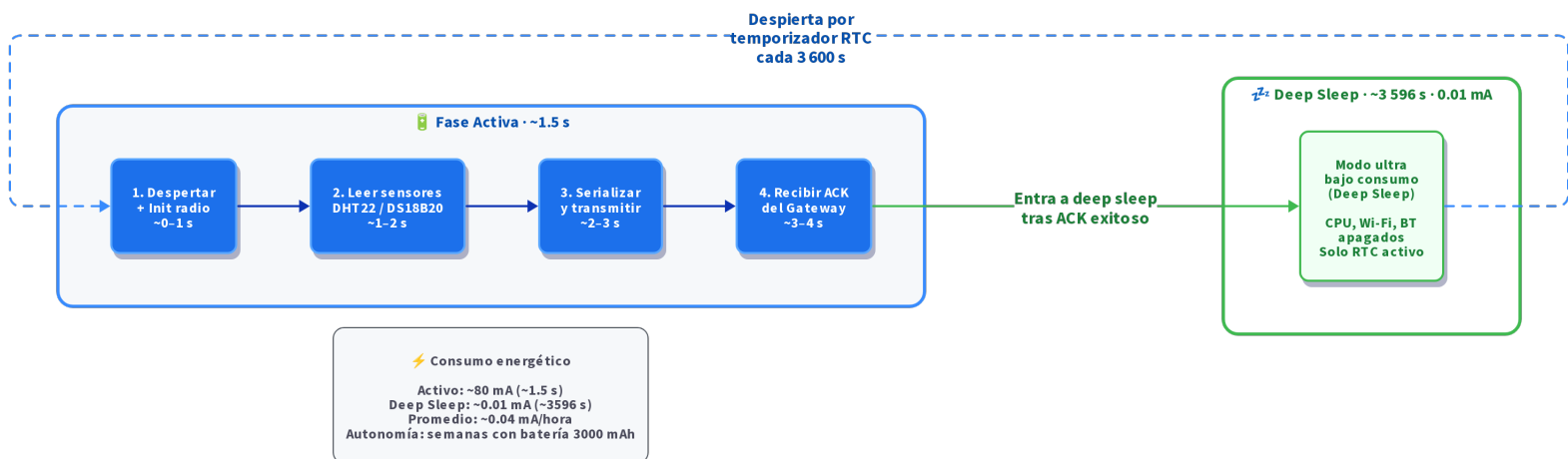


Figura A.8: Ciclo energético del nodo sensor: fase activa (~1,5 s) y Deep Sleep (~3596 s).

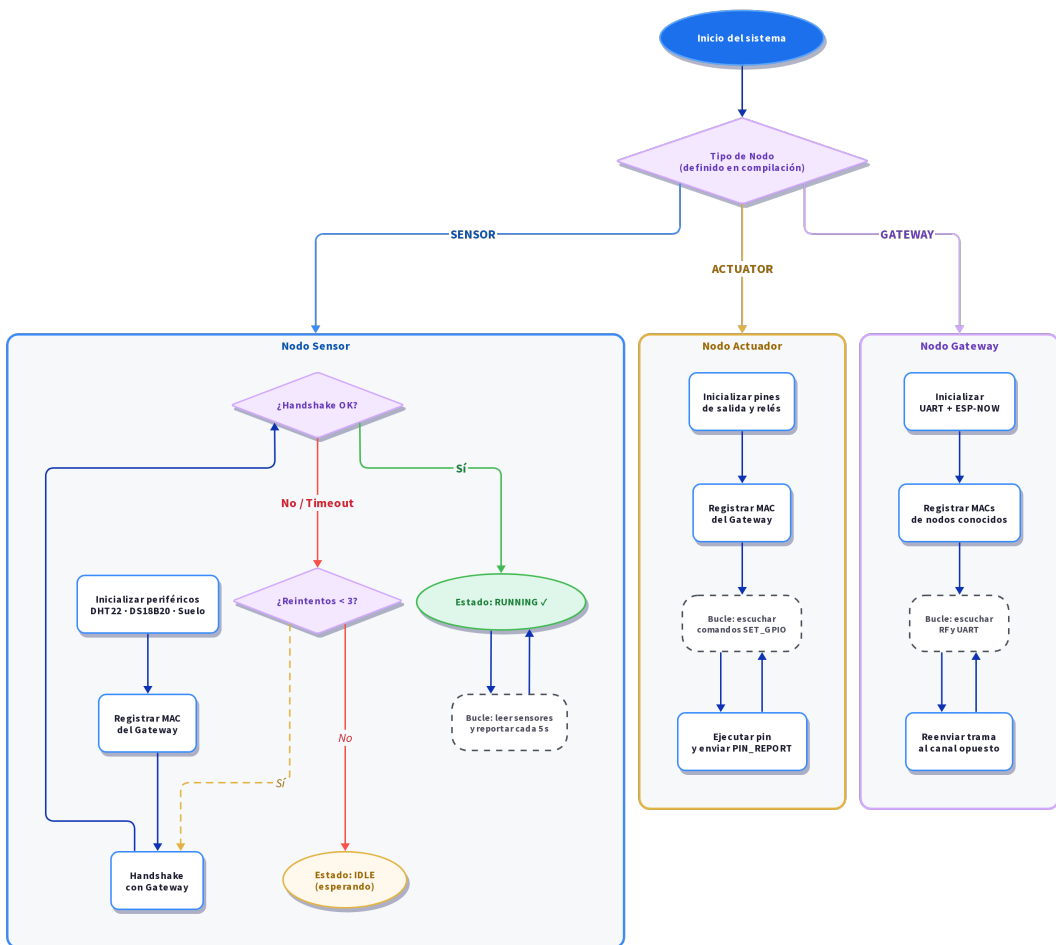


Figura A.9: Ciclo de vida de los nodos ESP32 según su rol: Sensor, Actuador o Gateway.

Apéndice B

Arquitectura del Servicio Raspberry Pi

Este anexo detalla la arquitectura del servicio Python ejecutado en la Raspberry Pi, que actúa como pasarela de borde (*edge gateway*) entre la red de nodos ESP32 y el servidor central. El servicio se ejecuta en un contenedor Docker con acceso directo al puerto serie GPIO (`/dev/serial10`) y gestiona cuatro tareas AsyncIO concurrentes: transporte UART, cliente WebSocket, servidor HTTP local (FastAPI en el puerto 8001) y el despachador de comandos. Los diagramas de este anexo cubren la arquitectura de componentes, los flujos de telemetría y comandos, el despliegue Docker y las tareas asíncronas.

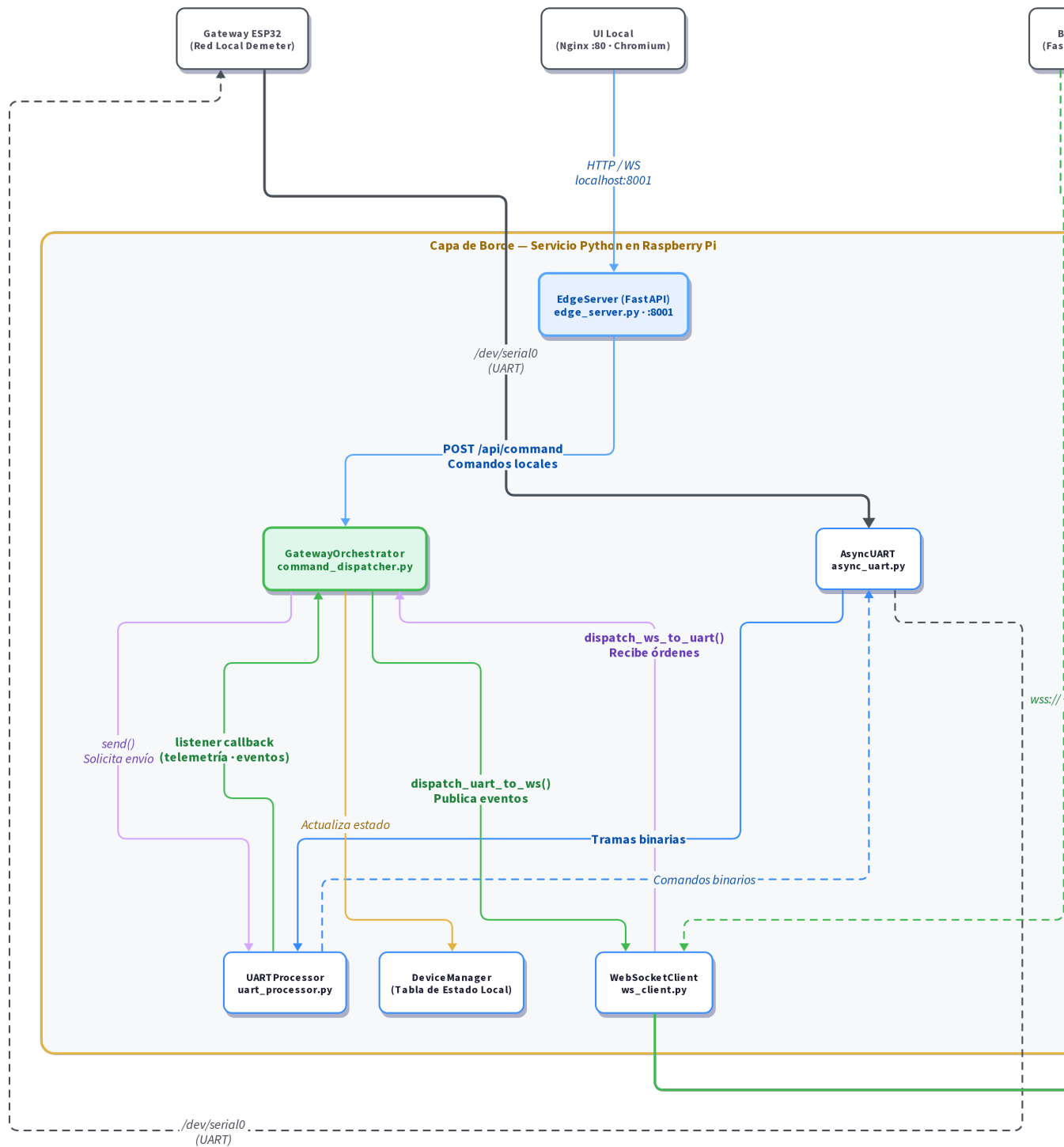


Figura B.1: Arquitectura de componentes del servicio Python en la Raspberry Pi (capa de borde).

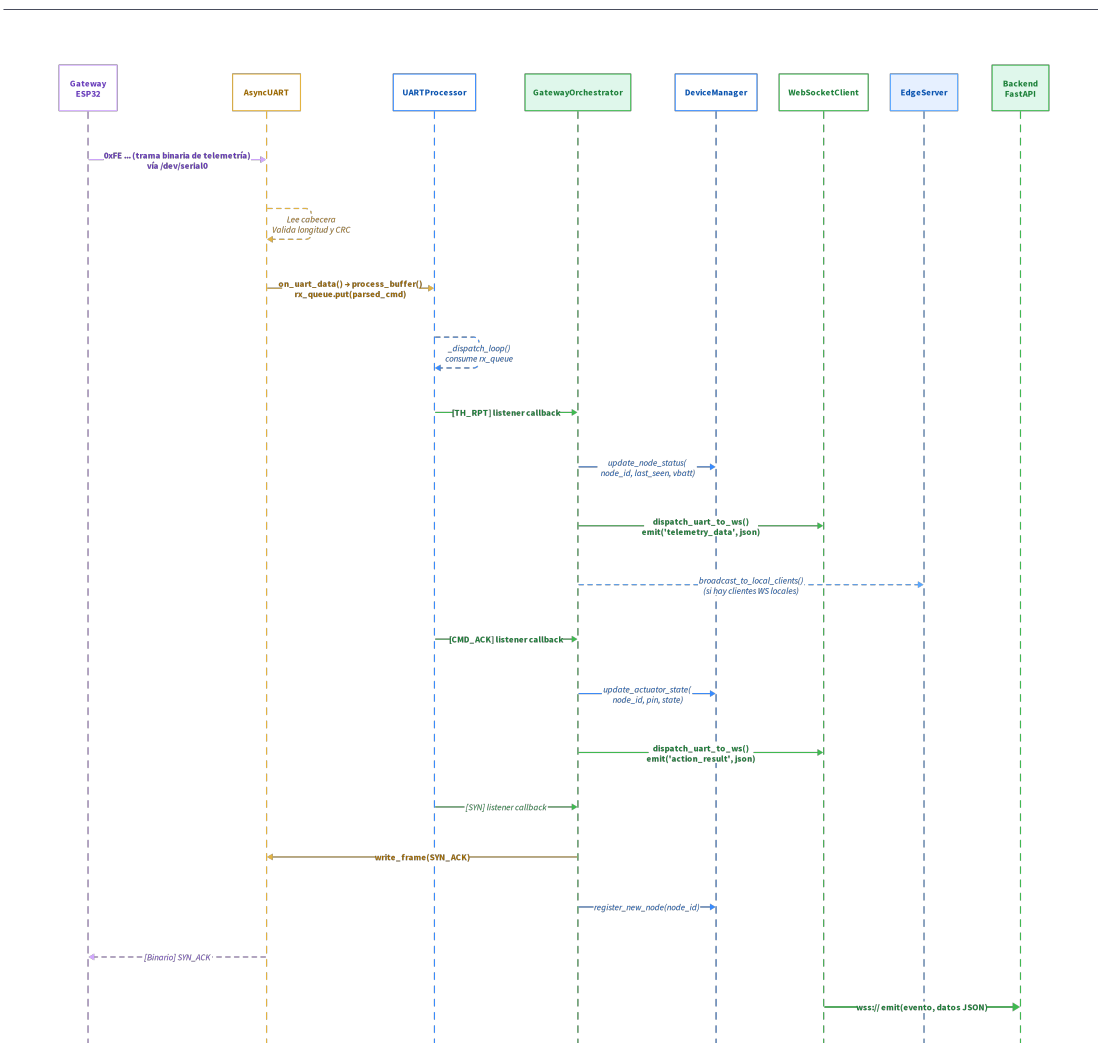


Figura B.2: Diagrama de secuencia: flujo ascendente de telemetría desde el Gateway hasta el Backend.

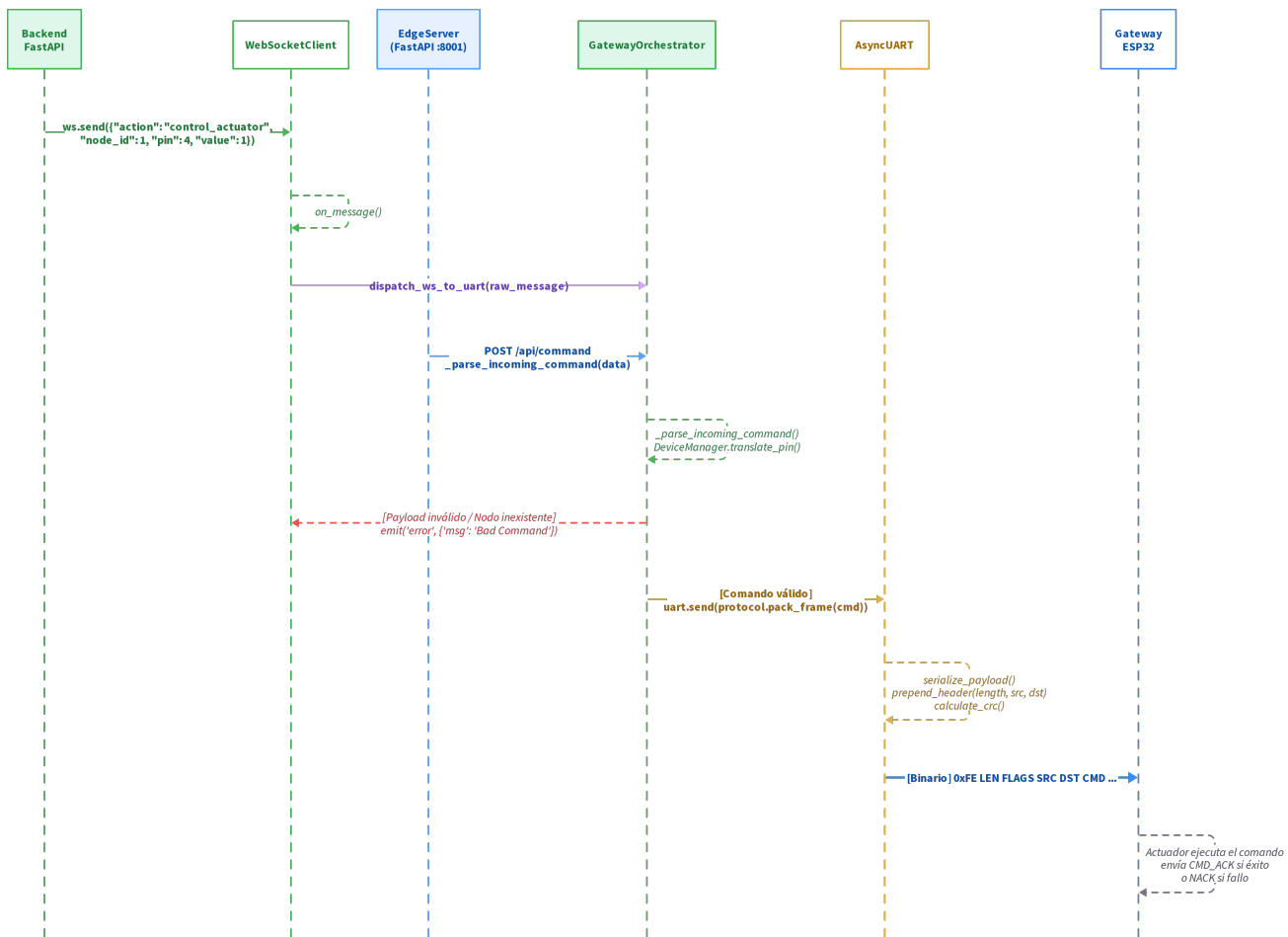


Figura B.3: Diagrama de secuencia: flujo descendente de comandos desde el Backend hasta el Gateway ESP32.

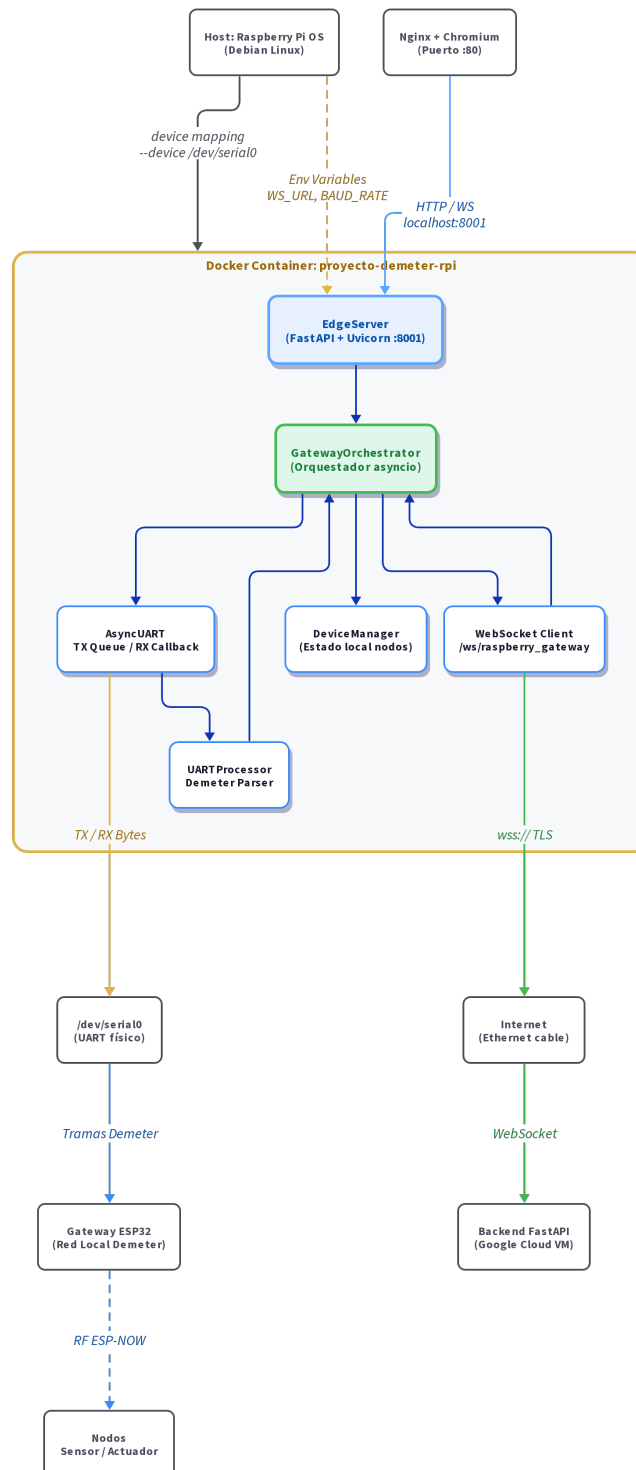


Figura B.4: Despliegue del servicio Raspberry Pi en contenedor Docker con sus conexiones externas.

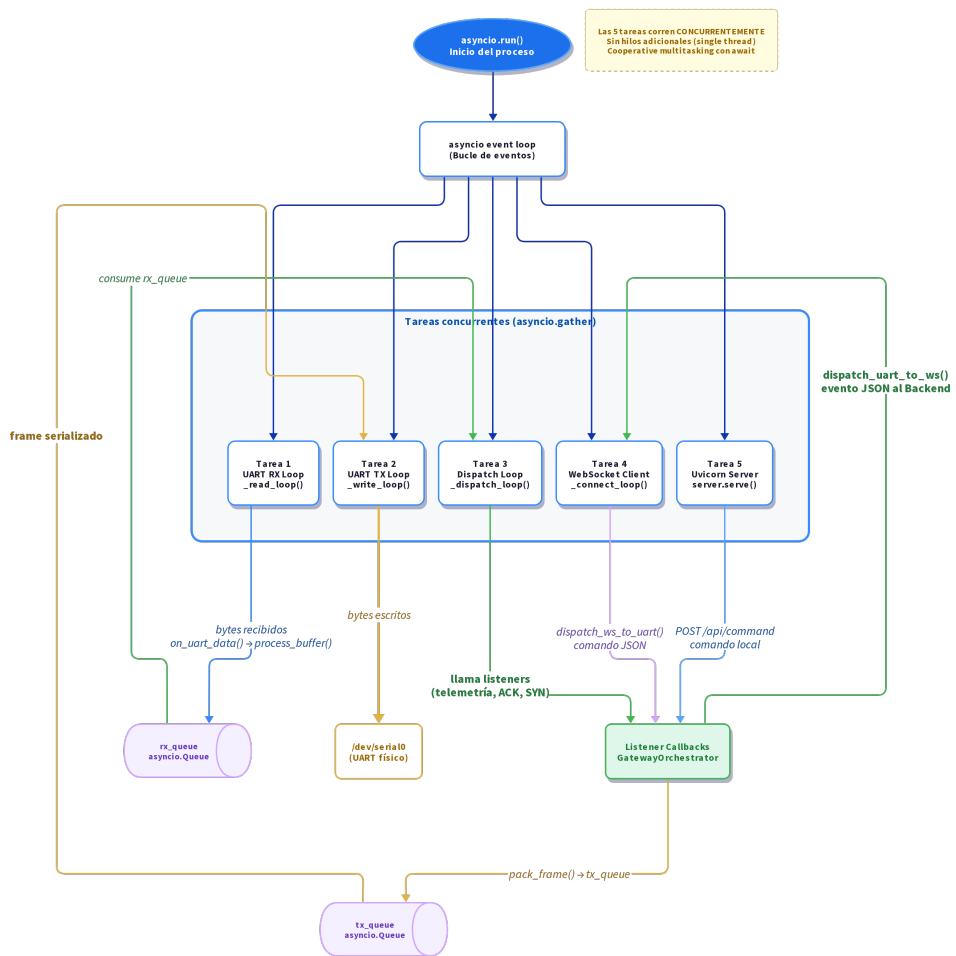


Figura B.5: Diagrama de las cuatro tareas AsyncIO concurrentes del servicio Raspberry Pi.

Apéndice C

Arquitectura del Backend

Este anexo presenta los diagramas de la arquitectura del servidor central desplegado en Google Cloud Platform. El backend se compone de seis contenedores Docker (API FastAPI, frontend Nginx, PostgreSQL, Redis, math-worker RQ y secuenciador) interconectados a través de la red privada **demeter-net**. La comunicación con la Raspberry Pi se realiza mediante WebSocket y el despacho de comandos utiliza el patrón Redis Pub/Sub descrito en el Capítulo [4](#).

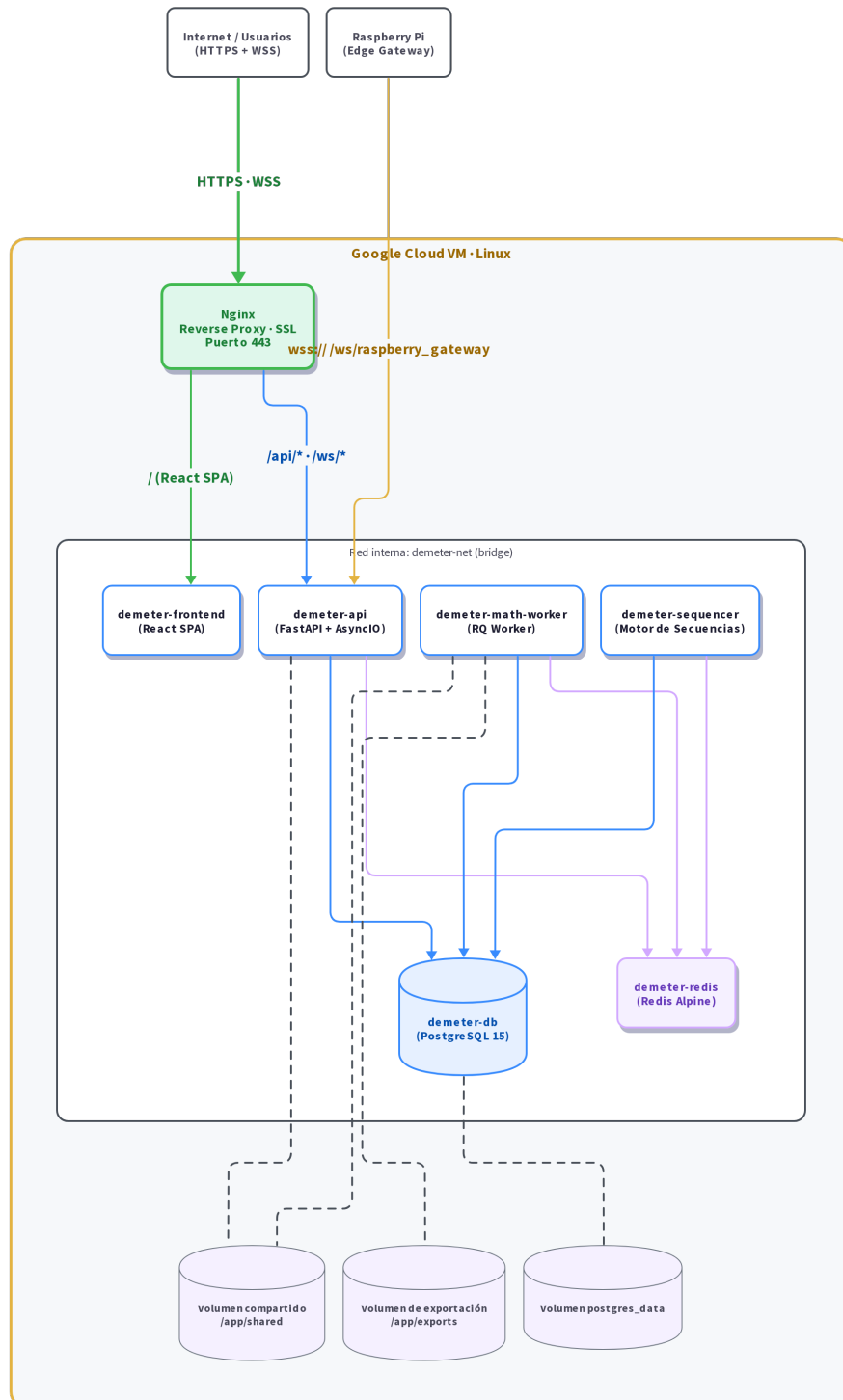


Figura C.1: Arquitectura de contenedores Docker del Backend en Google Cloud VM.

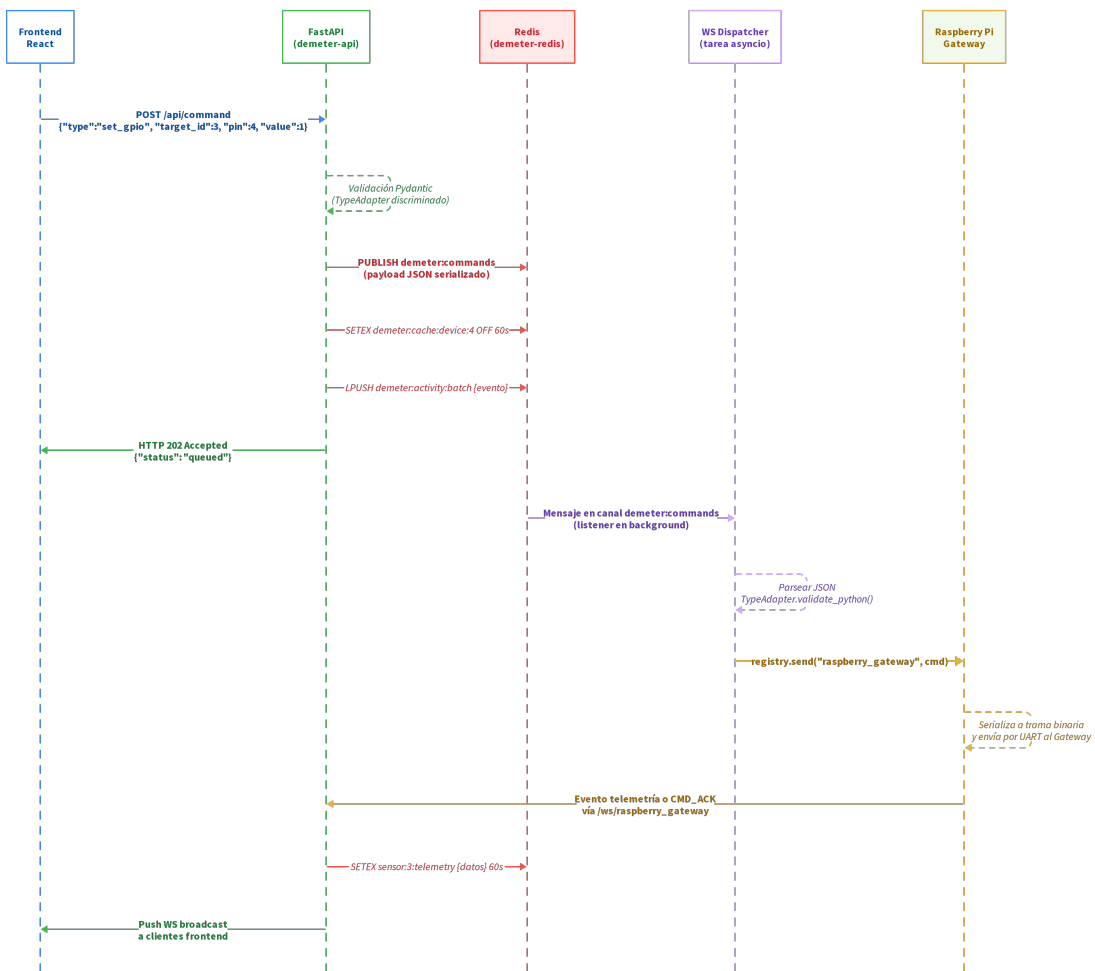


Figura C.2: Flujo de un comando de usuario a través del sistema Redis Pub/Sub hasta la Raspberry Pi.